



AMERICAN
UNIVERSITY
OF BEIRUT

Fall 2025

EECE 490: Introduction to Machine Learning

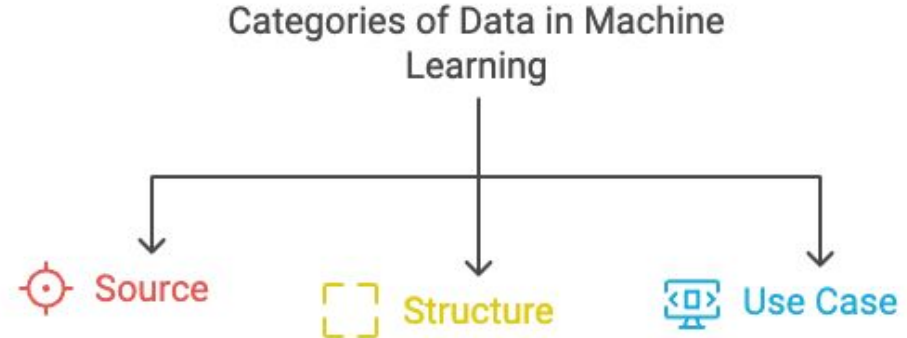
C2 - Preparing Data for Statistical Machine Learning Algorithms

Types of Data in Machine Learning

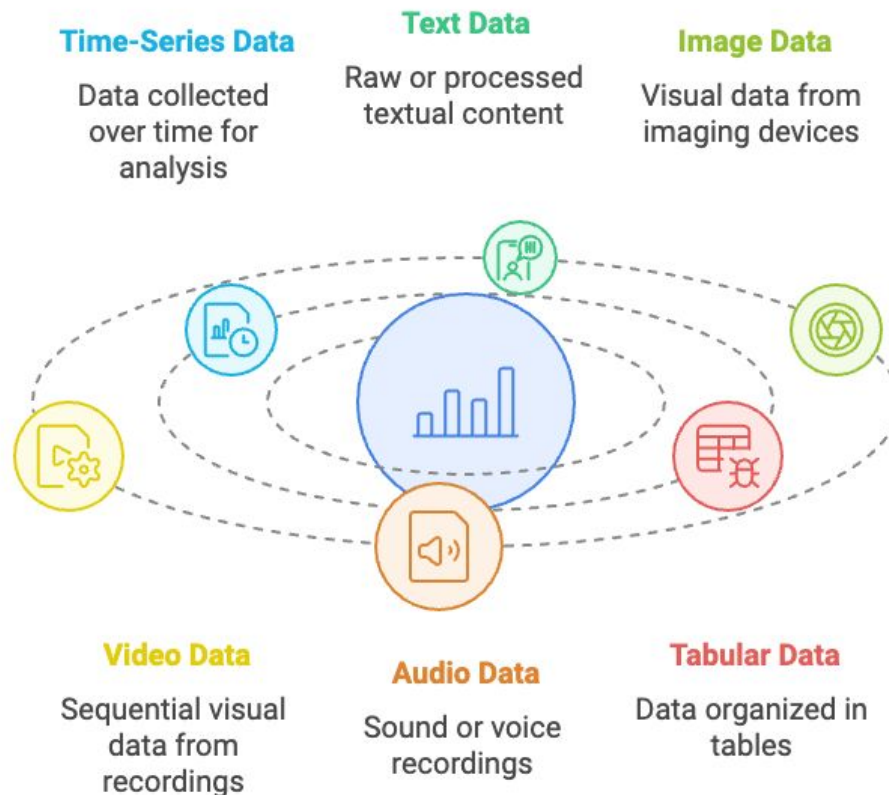
Categories of Data In Machine Learning

Data in machine learning studied from three different dimensions:

1. Data Source
2. Data Structure
3. Data Use Case

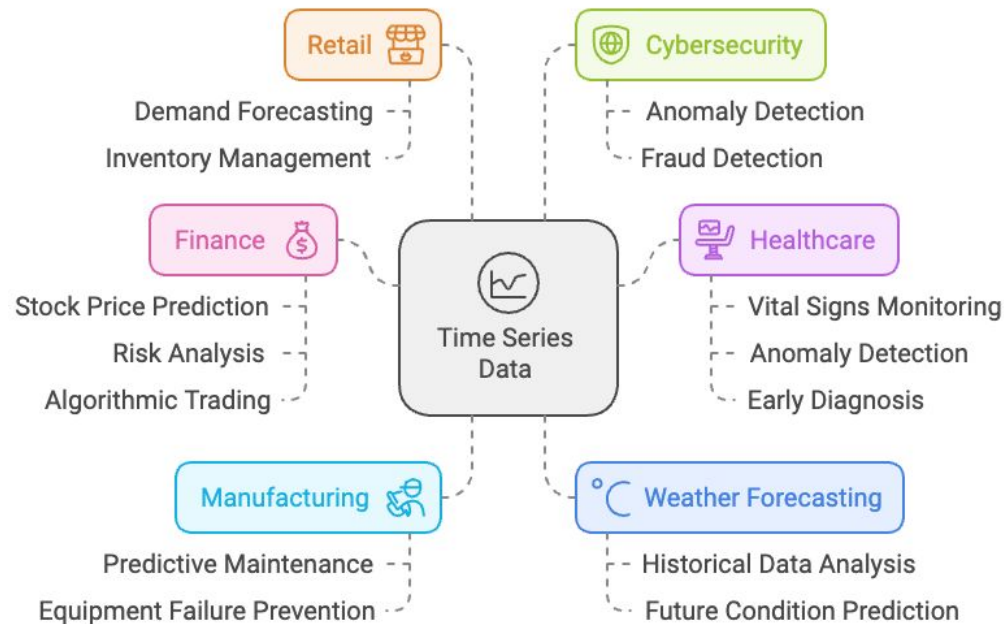
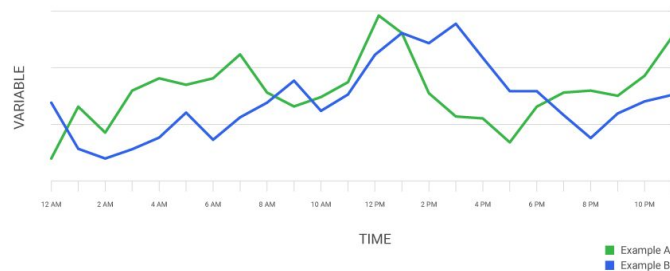


Data in ML based on sources



Time Series Data

Time series data is data collected over time at regular or irregular intervals.





Time Series Data Popular Real World Use Case

Netflix uses time series data to analyze:

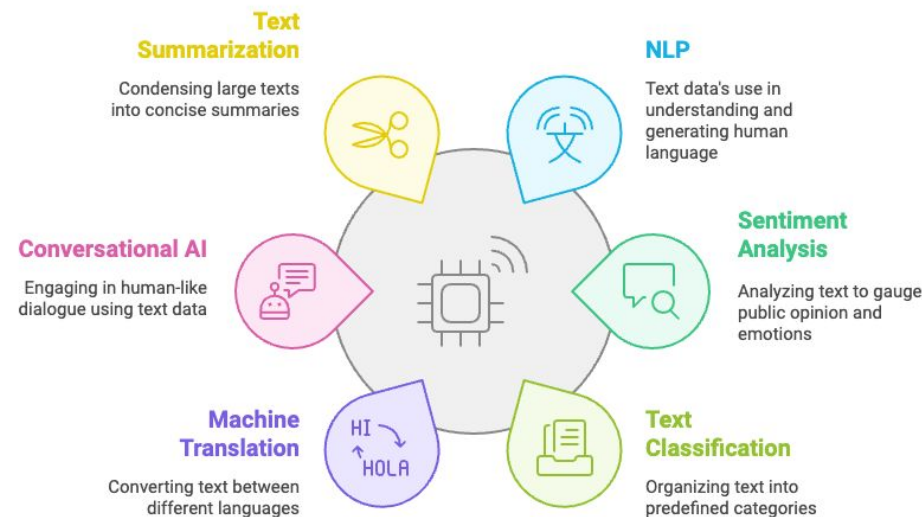
- User Viewing Patterns
- Content Engagement Over Time
- Personalized Recommendations
- Seasonal Trends
- User Retention Strategies



Text Data

Text data can be:

- Natural Language: any language that occurs naturally in a human community by a process of use, repetition, and change.
- Programming Language: Python, C++, Java,...
- Markup Language: HTML, XML,...
- Symbolic Text: mathematical equations
- Log Files: logs reported by software or hardware





Text Data Real World Use Case

AI personal assistants have gained extreme popularity in the last couple of years, helping automate or accelerate day-to-day work.



Image Data

Image data is visual data collected via cameras or other imaging devices.

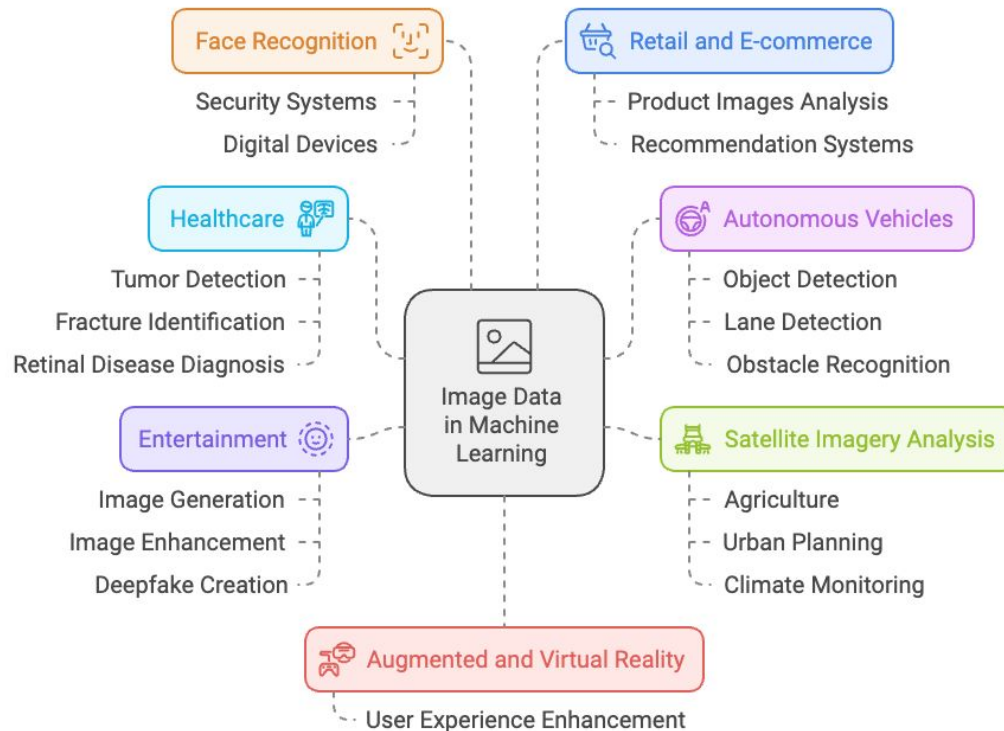
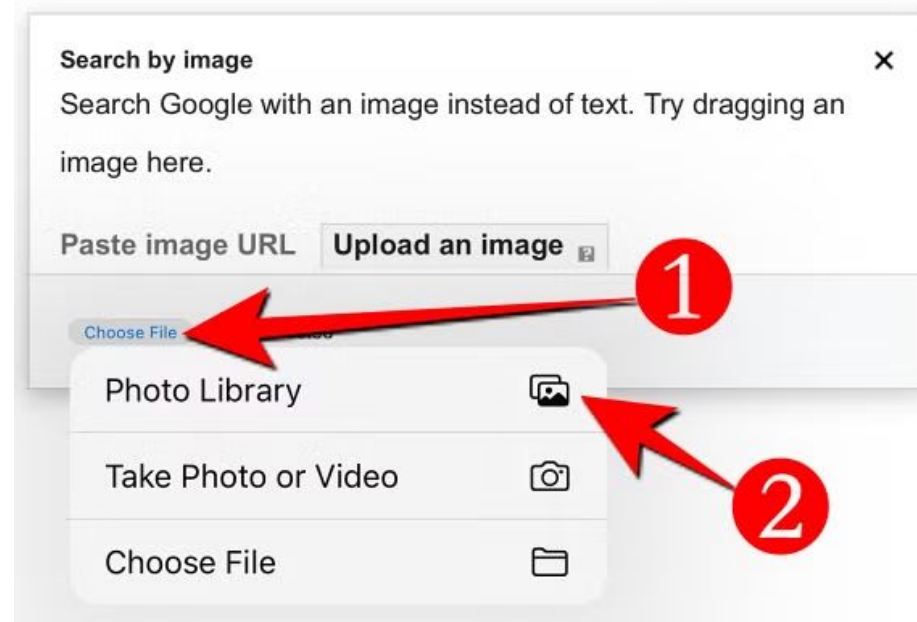


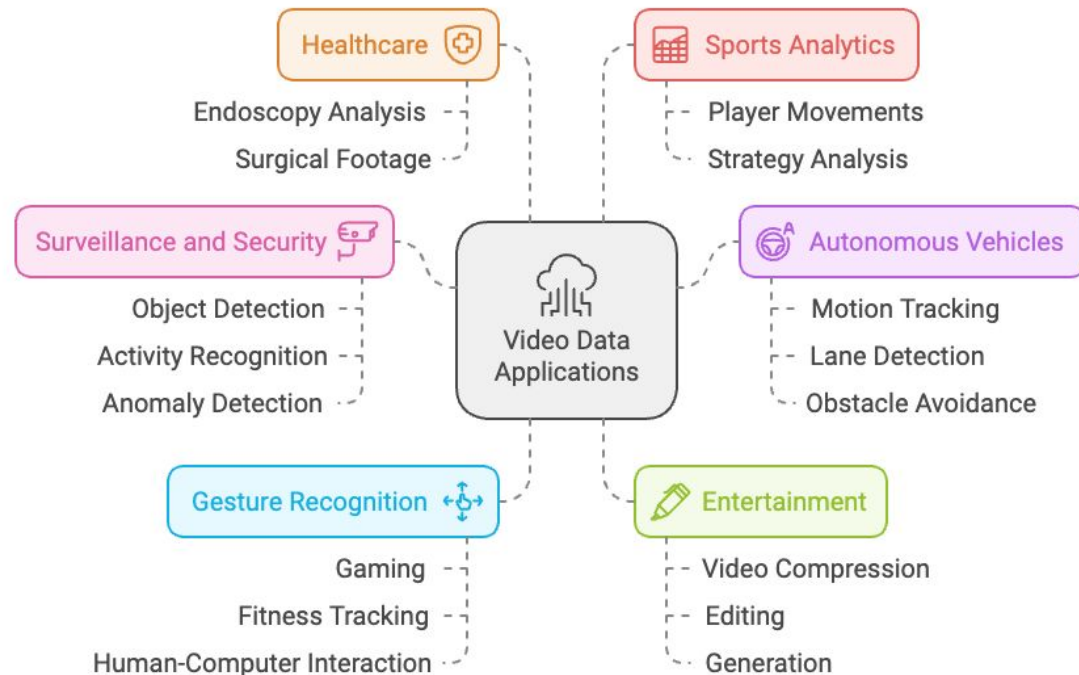
Image Data Real World Use Case

Google Image uploads uses ML to identify similar images available on the internet.



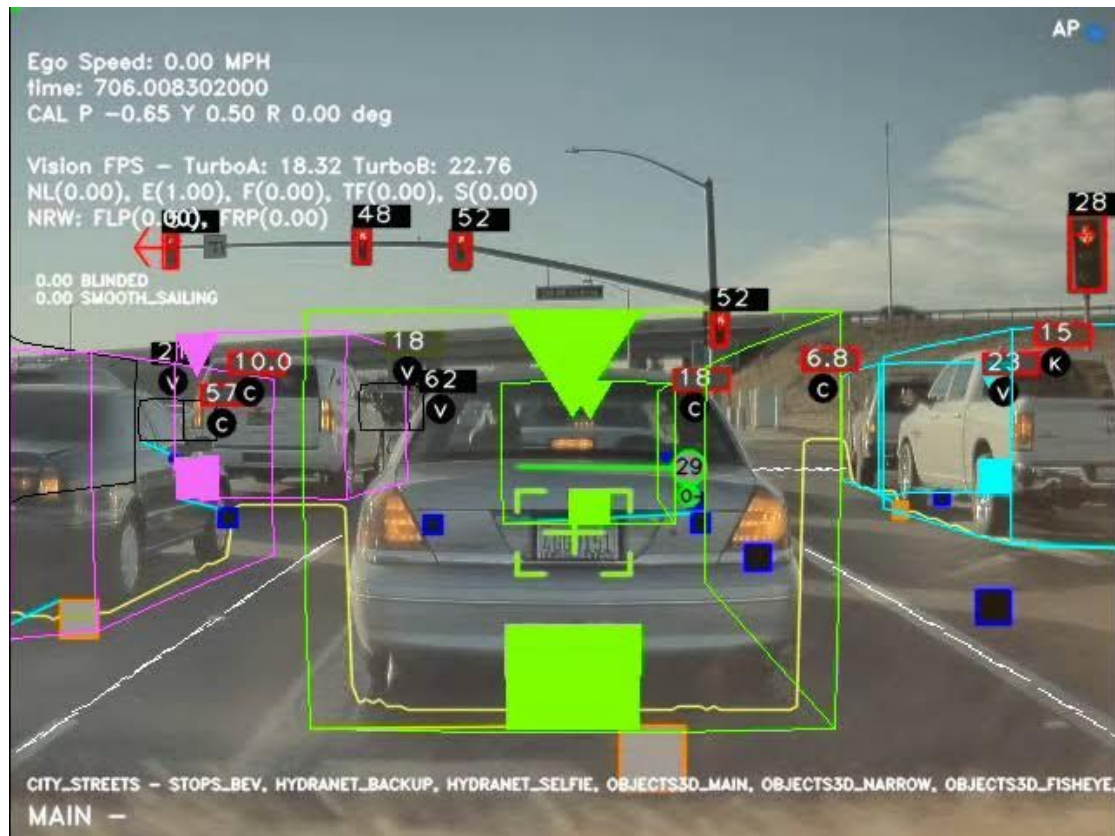
Video Data

Video data refers to a sequence of images (frames) captured and displayed in rapid succession over time, typically accompanied by audio.



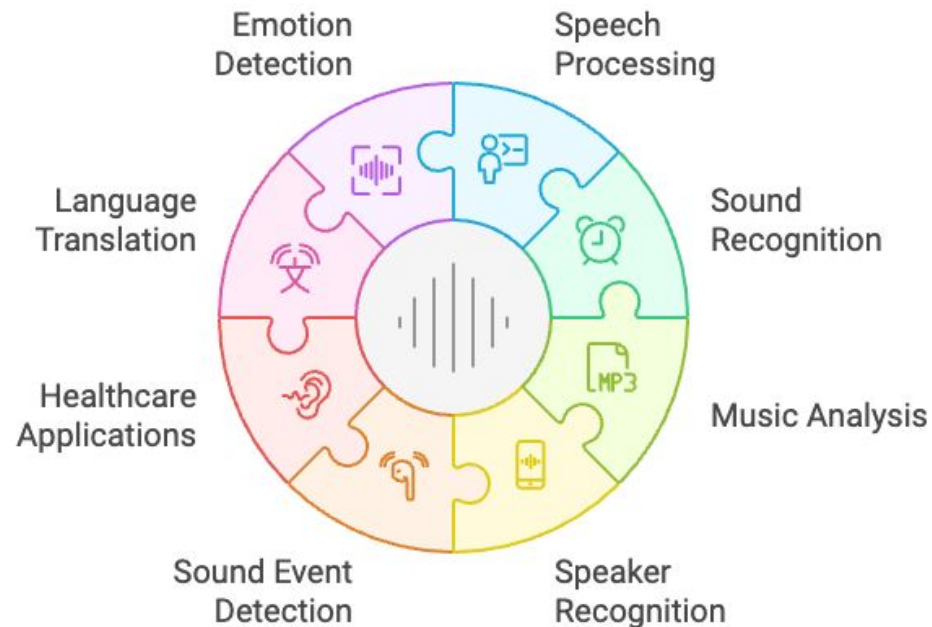
Video Data Real World Use Case

Tesla autopilot uses real-time footage captured through dashcams to identify surrounding objects.



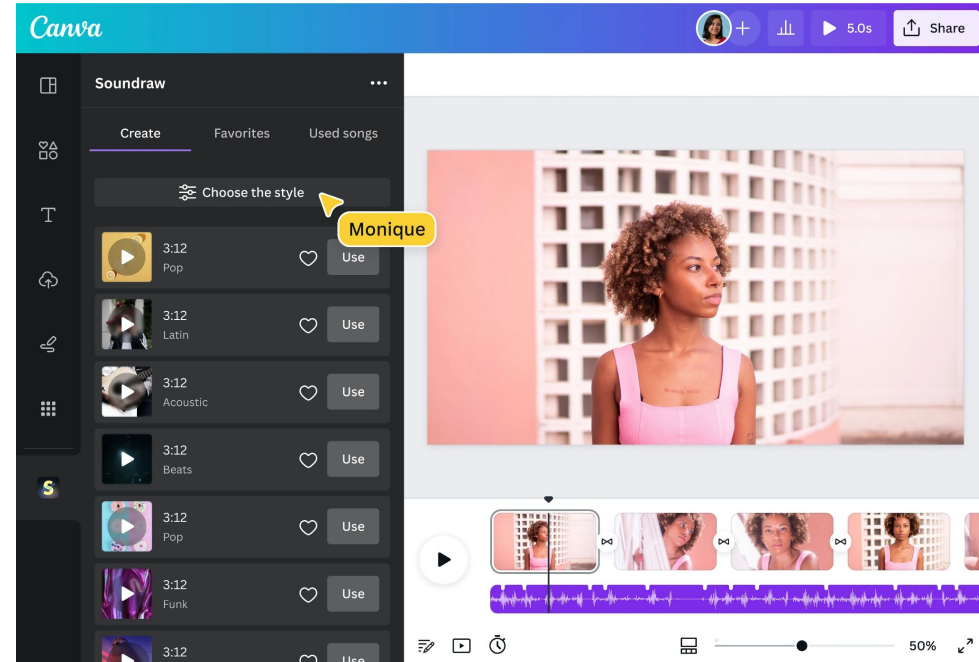
Audio Data

Audio data refers to sound signals captured in digital or analog form, typically represented as waveforms or frequency spectrums, encompassing speech, music, environmental sounds, or other acoustic information for analysis and processing.



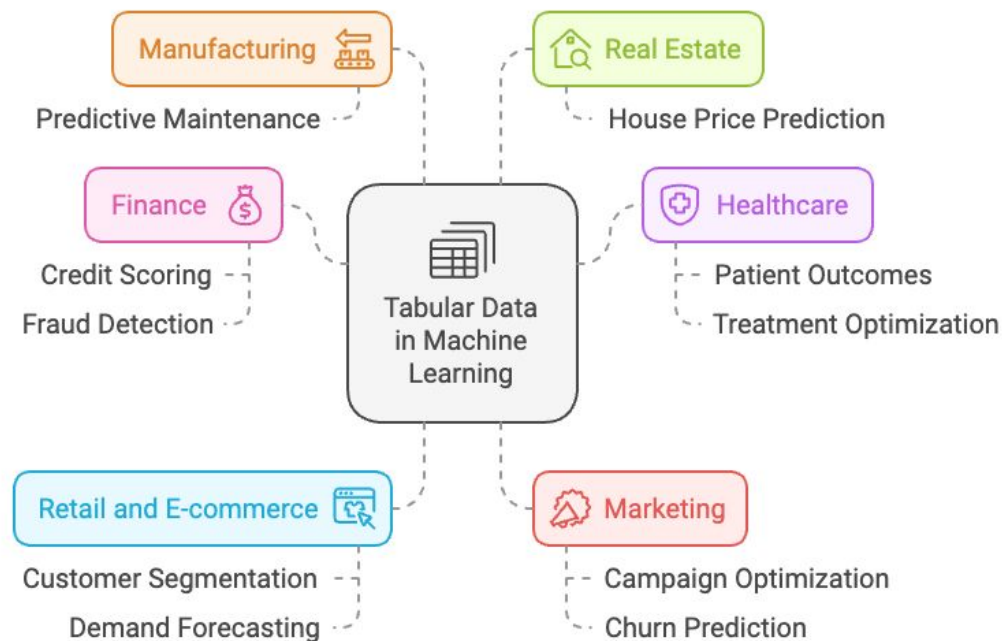
Audio Data Real World Use Case

Music generation is among the most popular use cases for audio data in AI.



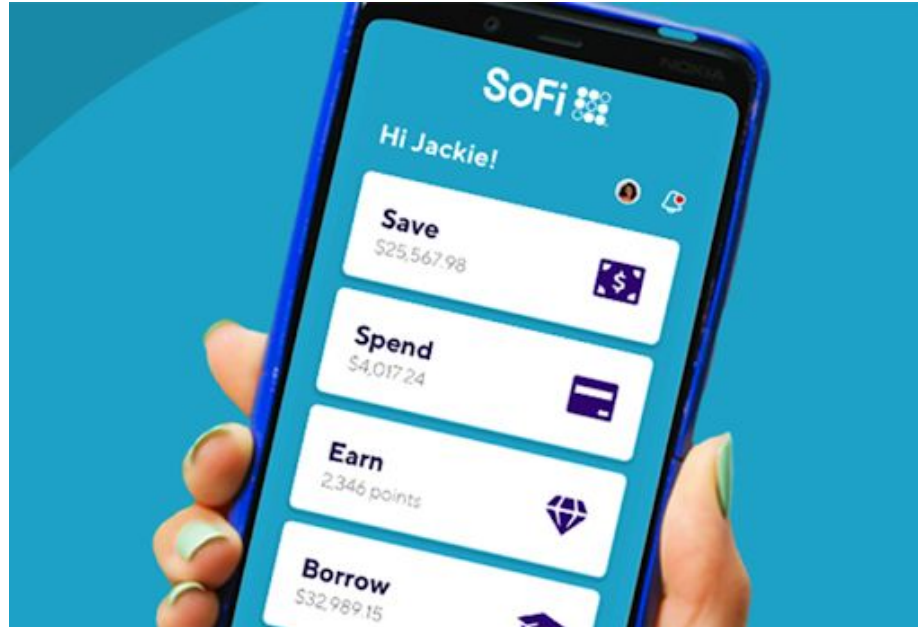
Tabular Data

Tabular data is structured data organized in rows and columns, resembling a table or spreadsheet, where each row represents an individual data instance (e.g., a customer, transaction, or observation), and each column represents a specific feature or attribute associated with the instances.

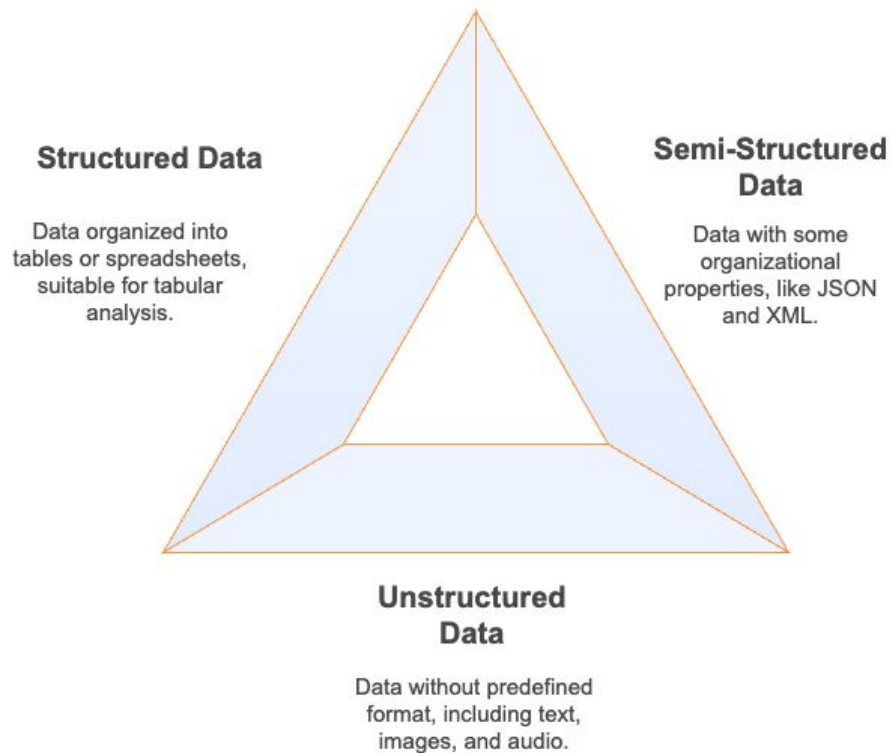


Tabular Data Real World Use Case

SoFi utilizes machine learning algorithms to assess applicants' creditworthiness by analyzing various data sources, including educational attainment, utility payments, insurance claims, and mobile phone usage.



Data in ML Based on Structure



Structured Data



Highly Organized

Follows a defined
schema

Easily Queryable

Can be accessed
and manipulated
using tools like
SQL or pandas

Storage Formats

CSV, Excel files,
relational
databases like
MySQL,
PostgreSQL



Example of structured data

Data to predict the price of a house.

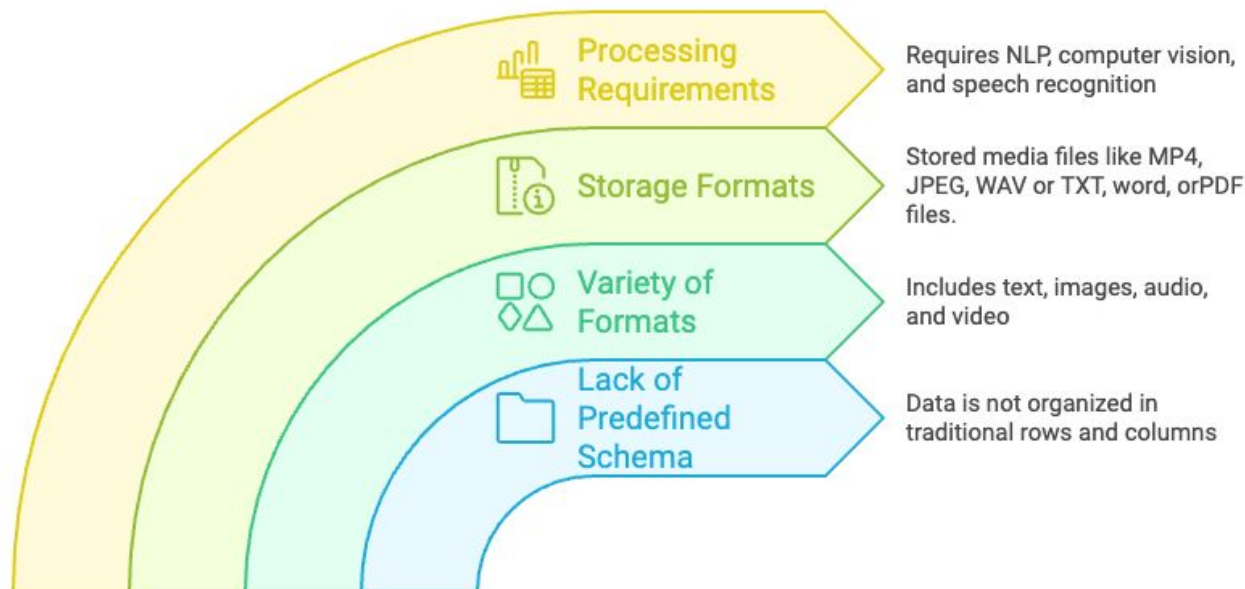
Housing

price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	hotwaterheating	airconditioning	parking	prefarea	furnishingstatus
13300000	7420	4	2	3	yes	no	no	no	yes	2	yes	furnished
12250000	8960	4	4	4	yes	no	no	no	yes	3	no	furnished
12250000	9960	3	2	2	yes	no	yes	no	no	2	yes	semi-furnished
12215000	7500	4	2	2	yes	no	yes	no	yes	3	yes	furnished
11410000	7420	4	1	2	yes	yes	yes	no	yes	2	no	furnished
10850000	7500	3	3	1	yes	no	yes	no	yes	2	yes	semi-furnished
10150000	8580	4	3	4	yes	no	no	no	yes	2	yes	semi-furnished
10150000	16200	5	3	2	yes	no	no	no	no	0	no	unfurnished
9870000	8100	4	1	2	yes	yes	yes	no	yes	2	yes	furnished
9800000	5750	3	2	4	yes	yes	no	no	yes	1	yes	unfurnished
9800000	13200	3	1	2	yes	no	yes	no	yes	2	yes	furnished
9681000	6000	4	3	2	yes	yes	yes	yes	no	2	no	semi-furnished
9310000	6550	4	2	2	yes	no	no	no	yes	1	yes	semi-furnished
9240000	3500	4	2	2	yes	no	no	yes	no	2	no	furnished
9240000	7800	3	2	2	yes	no	no	no	no	0	yes	semi-furnished

Unstructured Data



Understanding Unstructured Data



Example of unstructured data

Dataset for predicting handwritten digits.

label = 5



label = 0



label = 4



label = 1



label = 9



label = 2



label = 1



label = 3



label = 1



label = 4



label = 3



label = 5



label = 3



label = 6



label = 1



label = 7



label = 2



label = 8



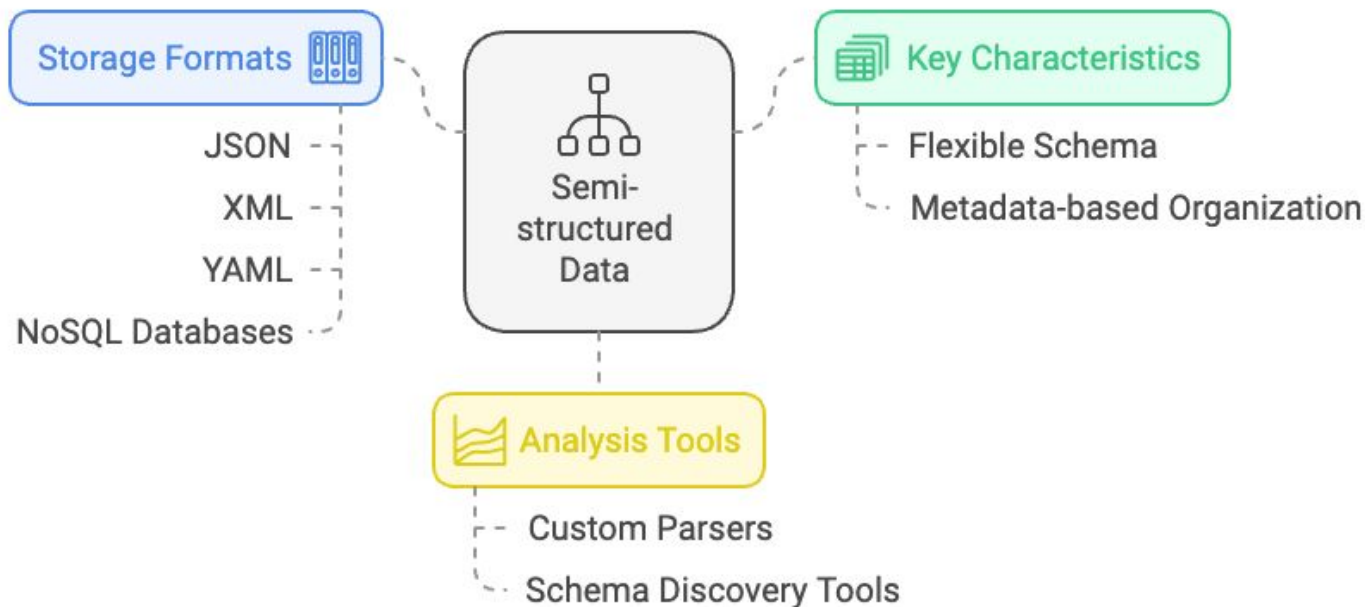
label = 6



label = 9



Semi-Structured Data

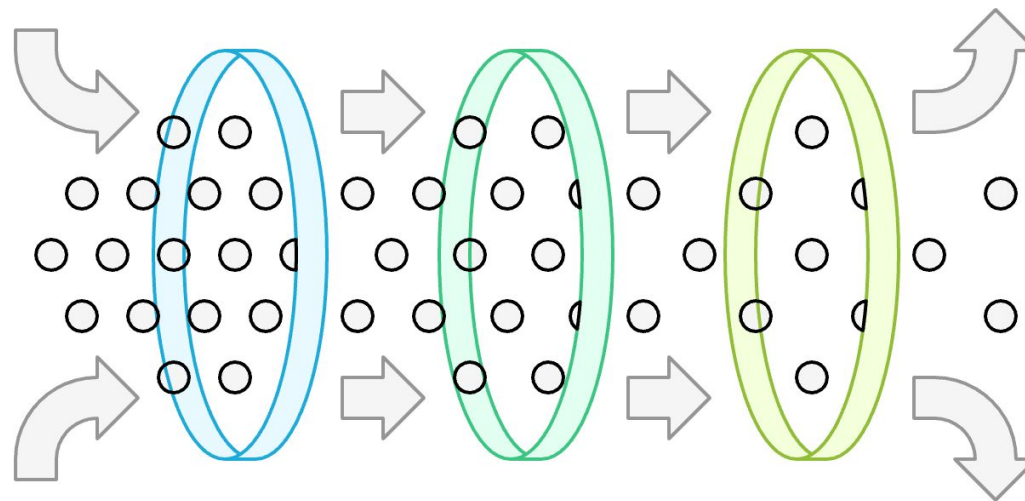


Semi-structured data example

Data to analyze sale patterns

```
1  {
2      "EMPLOYEES": {
3          "SALES": {
4              "648229": {
5                  "NAME" : "Olivia Johnson"
6                  "DOB"  : "1989-08-08"
7              },
8              "648666": {
9                  "NAME" : "Frank Mueller"
10                 "DOB"  : "1985-05-11"
11                 "MISC" : "On paternal leave from 2019-01-01 until 2020-01-01"
12             }
13         }
14     }
15 }
```

Data in ML based on purpose



Training Data

Model learns
patterns

Validation Data

Model
parameters
fine-tuned

Testing Data

Model
performance
evaluated

Training Data

Training data is fed into a machine learning **algorithm** to teach it how to perform a task or to analyze the data patterns. Once the algorithm is trained, it is called a machine learning **model**.



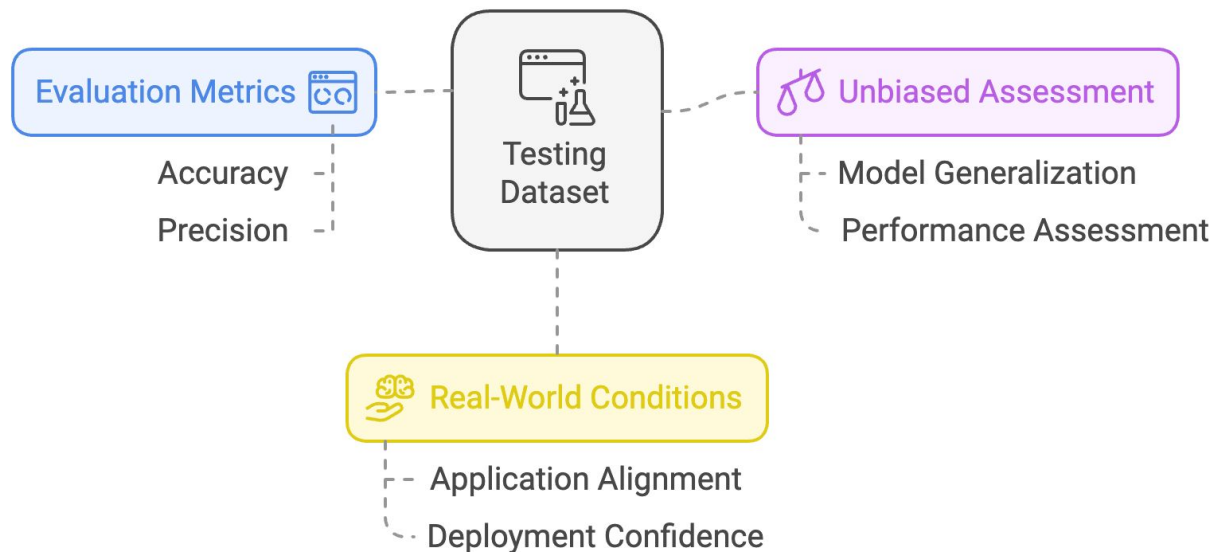
Validation Data

Validation dataset is used to check the model's performance during the training process. It allows us to choose which model 'settings' result in the highest accuracy.

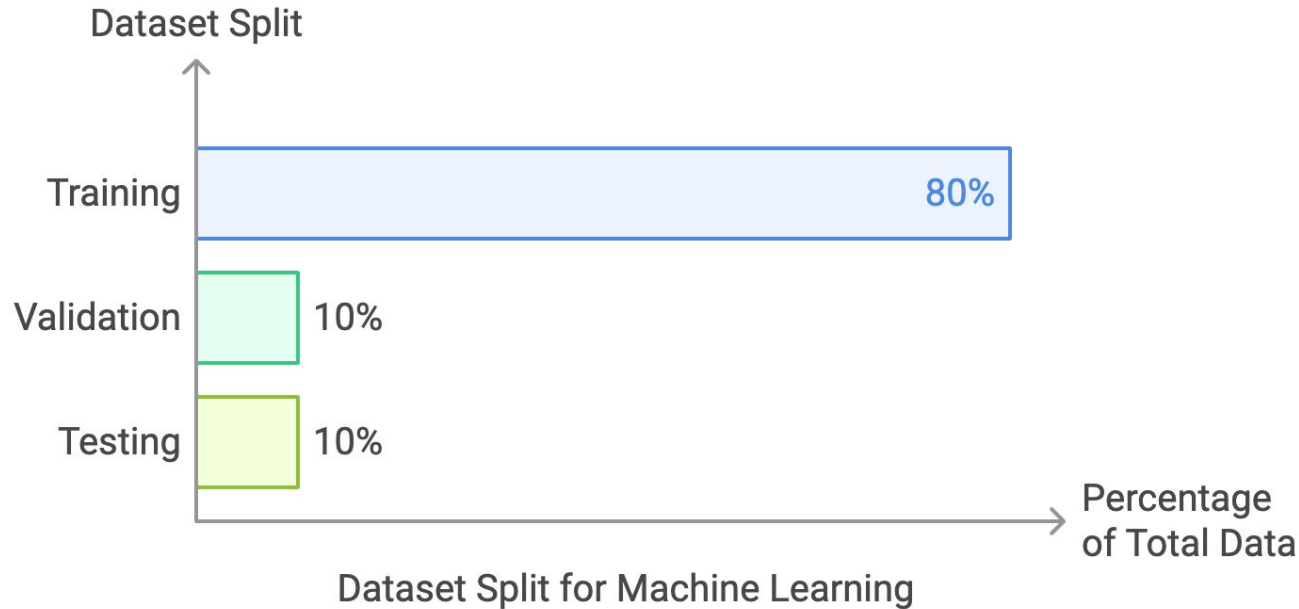


Testing Dataset

Testing dataset is used to check the performance of the model on data it **has not seen** during training.



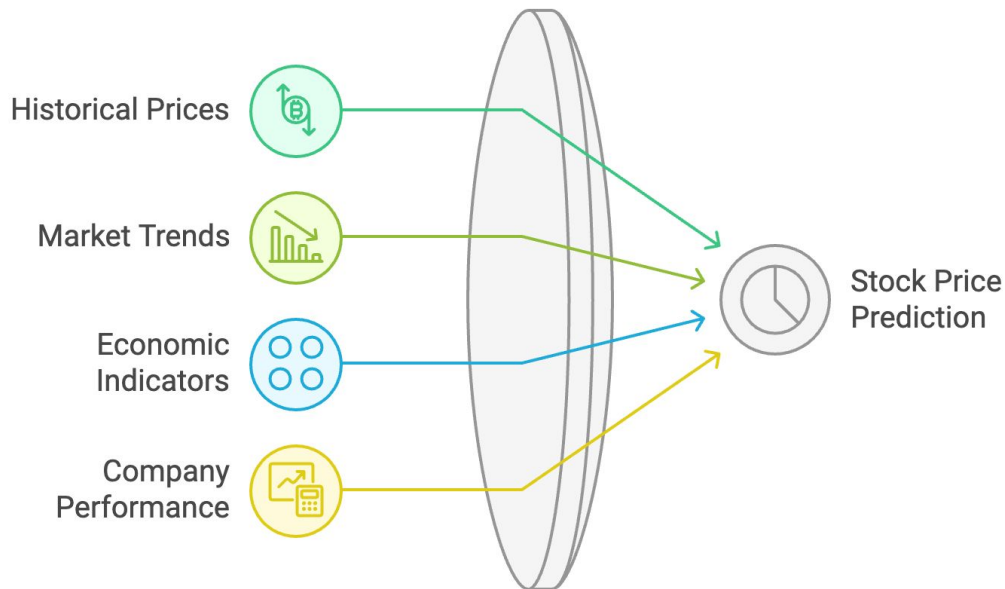
The Dataset Split



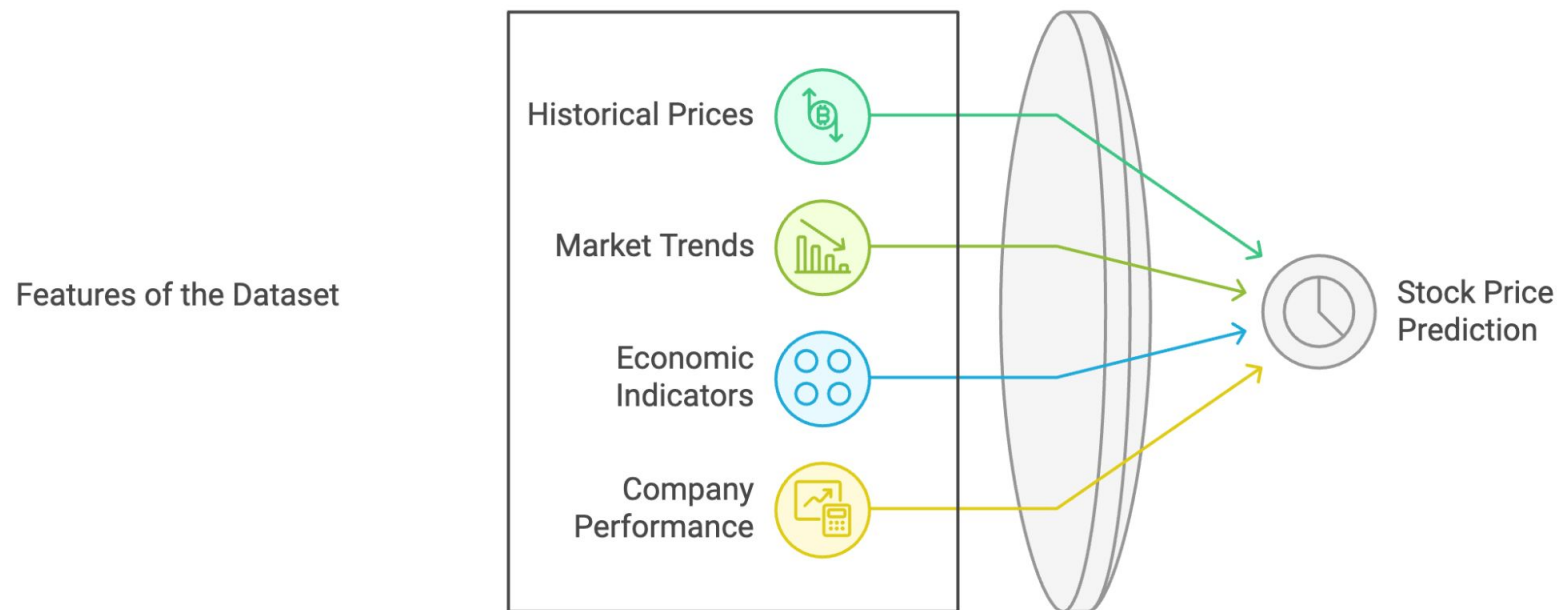
Features: Explicit and Implicit Information in the Dataset

Stock Price Prediction Example

Let's say we want to predict the price of a stock, what information would you look at to make this prediction?

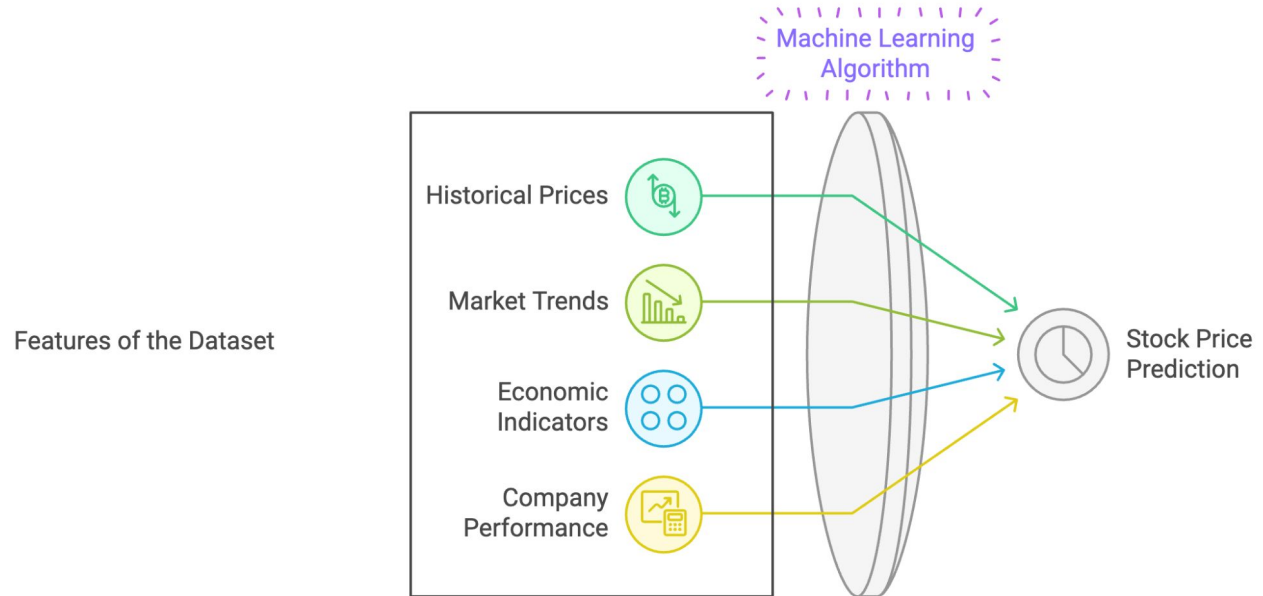


Stock Price Prediction Example



Stock Price Prediction Example

If we wanted to feed the machine learning algorithm training data to allow it to learn how to predict the prices of stock, the features would be the input to the algorithm.



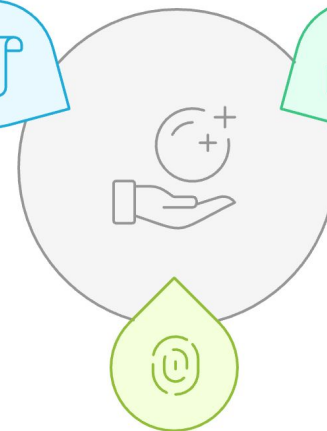
Features of a Machine Learning Dataset

Features are the **individual measurable properties** or characteristics of the data used by a machine learning model to make predictions or decisions.

Role of Features in Machine Learning

Algorithm Inputs

Data fed into models for processing



Measurable Properties

Quantifiable attributes that define data characteristics



Data Characteristics

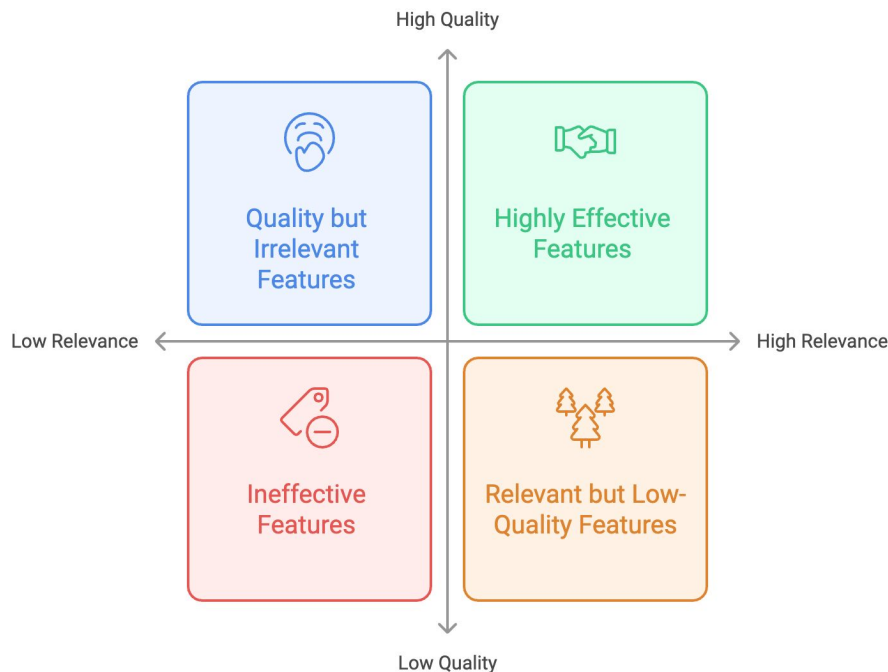
Distinctive features that represent data elements



Impact of Feature Properties on ML Model

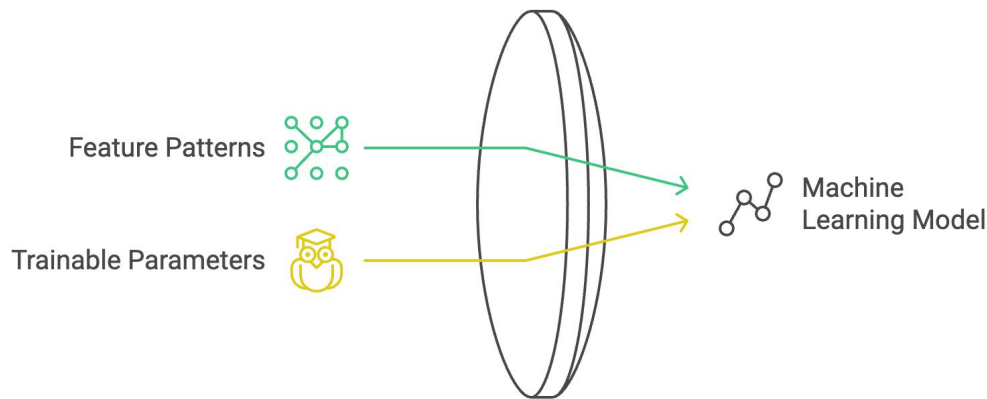
Not all features have an equal **contribution** to the model's prediction, but all features must be **relevant**.

Feature Quality and Relevance Impact on Predictions



Implicit Information within Features

The role of the machine learning model is to understand the underlying patterns and informations that the features hold. This implicit set of information is learned and stored within the model's **trainable parameters**.



Features in non-tabular data types

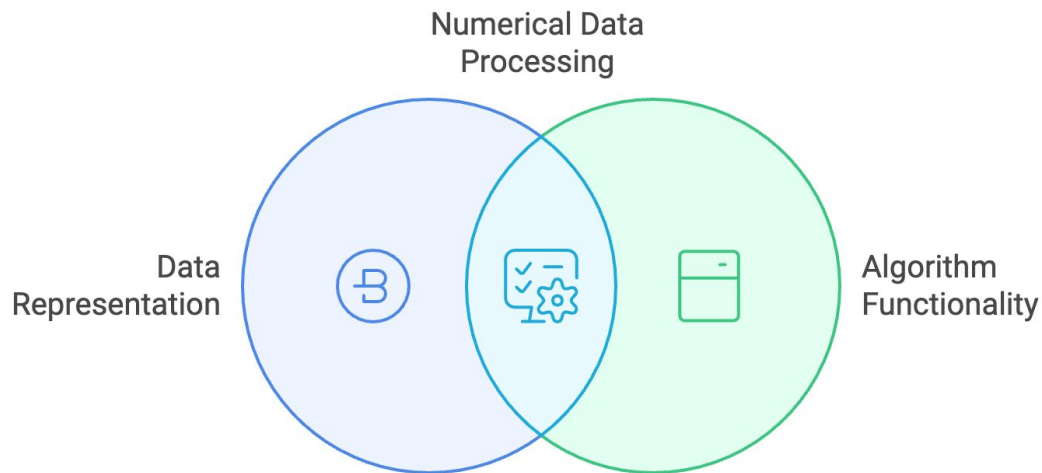
- **Text Data:** The features could only be the input text (generation purpose) or could include useful features like “most frequent words”, “document length”, (prediction purpose)..
- **Image Data:** The features could include only the color intensities inside the image or could include shape, depth of pixels, ...
- **Time Series and Audio Data:** The features could include only the timestamp and the value for each interval, but could include multiple other inferred features like spectral and pitch features.

How ML Algorithms Ingest Data

Numerical Representations

Machine learning models can only process data in **numerical representations** because mathematical computations underlie their functionality. Algorithms rely on numerical operations like **matrix multiplications, dot products, and gradient calculations**, which require data to be encoded as numbers.

The Role of Numerical Data in Machine Learning



Tabular Data

Numerical Representation of Tabular Data

The inputs and outputs of the machine learning model should be in numerical format. Let's say we want to predict if a person will get approved on a home loan.

Home Loan Approval Prediction

Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History	Property_Area	Loan_Status
LP001002	Male	No	0	Graduate	No	5849	0		360	1	Urban	Y
LP001003	Male	Yes	1	Graduate	No	4583	1508	128	360	1	Rural	N
LP001005	Male	Yes	0	Graduate	Yes	3000	0	66	360	1	Urban	Y
LP001006	Male	Yes	0	Not Graduate	No	2583	2358	120	360	1	Urban	Y
LP001008	Male	No	0	Graduate	No	6000	0	141	360	1	Urban	Y
LP001011	Male	Yes	2	Graduate	Yes	5417	4196	267	360	1	Urban	Y
LP001013	Male	Yes	0	Not Graduate	No	2333	1516	95	360	1	Urban	Y
LP001014	Male	Yes	3	Graduate	No	3036	2504	158	360	0	Semiurban	N
LP001018	Male	Yes	2	Graduate	No	4006	1526	168	360	1	Urban	Y
LP001020	Male	Yes	1	Graduate	No	12841	10968	349	360	1	Semiurban	N
LP001024	Male	Yes	2	Graduate	No	3200	700	70	360	1	Urban	Y
LP001027	Male	Yes	2	Graduate		2500	1840	109	360	1	Urban	Y
LP001028	Male	Yes	2	Graduate	No	3073	8106	200	360	1	Urban	Y

Numerical Representation of Tabular Data

We have two types of features (including output): **Numeric** and **Categorical**

Home Loan Approval Prediction

Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History	Property_Area	Loan_Status
LP001002	Male	No	0	Graduate	No	5849	0		360	1	Urban	Y
LP001003	Male	Yes	1	Graduate	No	4583	1508	128	360	1	Rural	N
LP001005	Male	Yes	0	Graduate	Yes	3000	0	66	360	1	Urban	Y
LP001006	Male	Yes	0	Not Graduate	No	2583	2358	120	360	1	Urban	Y
LP001008	Male	No	0	Graduate	No	6000	0	141	360	1	Urban	Y
LP001011	Male	Yes	2	Graduate	Yes	5417	4196	267	360	1	Urban	Y
LP001013	Male	Yes	0	Not Graduate	No	2333	1516	95	360	1	Urban	Y
LP001014	Male	Yes	3	Graduate	No	3036	2504	158	360	0	Semiurban	N
LP001018	Male	Yes	2	Graduate	No	4006	1526	168	360	1	Urban	Y
LP001020	Male	Yes	1	Graduate	No	12841	10968	349	360	1	Semiurban	N
LP001024	Male	Yes	2	Graduate	No	3200	700	70	360	1	Urban	Y
LP001027	Male	Yes	2	Graduate		2500	1840	109	360	1	Urban	Y
LP001028	Male	Yes	2	Graduate	No	3073	8106	200	360	1	Urban	Y

Numerical Representation of Tabular Data

The goal is to transform all the features to a numeric representation. Obviously, we do not need to change the features that are already numeric, so let's explore the categorical features.

First, we need to import our data from kaggle.

```
import kagglehub

path = kagglehub.dataset_download("rishikeshkonapure/home-loan-approval")

print("Path to dataset files:", path)
```

Numerical Representation of Tabular Data

After importing our data we need to load it. From the dataset card on kaggle, we see that we have **two csv files**: one for training, and one for testing. We will work on the training file.

To load the training csv file, we will use the **pandas library**.

```
import pandas as pd

home_loan_prediction = pd.read_csv(path + "/loan_sanction_train.csv")
home_loan_prediction
```

1 to 25 of 614 entries  

Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History	Property_Area	Loan_Status
LP001002	Male	No	0	Graduate	No	5849	0.0	NaN	360.0	1.0	Urban	Y
LP001003	Male	Yes	1	Graduate	No	4583	1508.0	128.0	360.0	1.0	Rural	N
LP001005	Male	Yes	0	Graduate	Yes	3000	0.0	66.0	360.0	1.0	Urban	Y
LP001006	Male	Yes	0	Not Graduate	No	2583	2358.0	120.0	360.0	1.0	Urban	Y
LP001008	Male	No	0	Graduate	No	6000	0.0	141.0	360.0	1.0	Urban	Y

Numerical Representation of Tabular Data

Now that we have our dataset, let's select the categorical features. We can know which features are categorical by using the `.info()` function in pandas that would tell us the object type of the feature.

```
home_loan_prediction.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 614 entries, 0 to 613
Data columns (total 13 columns):
#   Column              Non-Null Count  Dtype  
---  -
0   Loan_ID              614 non-null    object  
1   Gender               601 non-null    object  
2   Married              611 non-null    object  
3   Dependents           599 non-null    object  
4   Education            614 non-null    object  
5   Self_Employed        582 non-null    object  
6   ApplicantIncome      614 non-null    int64   
7   CoapplicantIncome    614 non-null    float64  
8   LoanAmount           592 non-null    float64  
9   Loan_Amount_Term     600 non-null    float64  
10  Credit_History        564 non-null    float64  
11  Property_Area         614 non-null    object  
12  Loan_Status          614 non-null    object  
dtypes: float64(4), int64(1), object(8)
memory usage: 62.5+ KB
```

Numerical Representation of Tabular Data

Alternatively, we can use the `.select_dtypes()` to select the features that have the 'object' type and print a list of their names.

```
categorical_features = home_loan_prediction.select_dtypes(include=['object']).columns.tolist()
```

```
categorical_features
```

```
['Loan_ID',  
 'Gender',  
 'Married',  
 'Dependents',  
 'Education',  
 'Self_Employed',  
 'Property_Area',  
 'Loan_Status']
```

Numerical Representation of Tabular Data

The process of transforming a categorical feature into its numeric representation is called **‘encoding’**.

Encoding Technique	Description	Advantages	Disadvantages
Label Encoding	Assigns each unique category in a feature a numerical value, often incrementally starting from 0.	- Simple and easy to implement - Suitable for ordinal data	- Introduces arbitrary ordinal relationships for nominal data - May not work well with outliers
One-Hot Encoding	Converts each category into binary columns (0 or 1), with each column representing a unique category.	- Suitable for nominal data - Avoids introducing ordinal relationships - Maintains information on the values of each variable	- Can lead to increased dimensionality and sparsity - May cause overfitting with many categories or small sample sizes
Ordinal Encoding	Assigns integers to categories in a sequential order, preserving their inherent rank or hierarchy.	- Preserves the order of categories - Suitable for ordinal data	- Not suitable for nominal data - Assumes equal spacing between categories, which may not be true
Target Encoding	Replaces categories with the mean of the target variable for each category, based on the dataset.	- Can improve model performance by incorporating target information - Suitable for high-cardinality features	- Prone to overfitting, especially with small datasets - Requires careful handling to avoid data leakage

Examples of encoding methods

Feature: **Color of a rose**: Red, White, Yellow, Pink

1. **Label Encoding**: 2, 3, 1, 0 (done in alphabetical order unless specified otherwise)
2. **One-hot encoding**: [1, 0, 0, 0], [0, 1, 0, 0], [0, 0, 1, 0], [0, 0, 0, 1]
3. **Ordinal Encoding**: 0, 1, 2, 3 (according to order of occurrence or hierarchy)
4. **Target Encoding**: 0.8, 0.7, 0.9, 0.6 (mean target value, e.g., price or rating, calculated for each category).

Numerical Representation of Tabular Data

Let's explore how we can use each encoding type in python code for the 'property area' feature in our dataset. The first thing we need to do is take a look at the 'categories' of this feature and their values.

```
unique_values_of_property_area = home_loan_prediction['Property_Area'].unique()  
unique_values_of_property_area  
  
array(['Urban', 'Rural', 'Semiurban'], dtype=object)
```




Numerical Representation of Tabular Data

Using label encoding:

```
from sklearn.preprocessing import LabelEncoder

label_encoder = LabelEncoder() # initialize the label encoder class
label_encoded_column = label_encoder.fit_transform(home_loan_prediction['Property_Area'])
print(label_encoded_column)
```

```
[2 0 2 2 2 2 2 1 2 1 2 2 2 0 2 2 2 2 0 2 2 2 1 0 1 1 1 2 2 1 2 2 0 1 0 2 2
 1 2 1 2 2 2 1 2 2 2 2 2 1 1 1 1 2 2 1 1 0 2 2 2 2 0 0 1 1 2 2 2 1 2 1 1 1
 1 2 2 2 1 1 1 1 2 1 2 1 1 1 2 1 1 1 2 1 1 2 1 1 1 2 1 2 1 2 2 0 2 1 2
 1 0 1 1 0 1 2 0 2 0 1 1 1 0 0 0 0 2 0 2 2 1 1 1 1 0 2 1 0 0 2 1 1 2 1 2 2
 0 1 0 0 2 0 2 1 0 2 0 1 1 2 1 0 2 0 0 0 1 1 0 2 0 1 1 0 0 1 1 2 2 0 1 1 1
 1 0 0 0 0 0 1 2 1 0 1 0 2 1 2 1 1 2 2 1 1 2 0 2 1 1 1 2 0 2 1 0 1 1 1 2 1
 1 1 1 0 2 1 1 0 1 0 0 1 1 0 2 2 0 1 0 2 2 0 1 2 2 2 1 2 1 2 0 1 2 0 0 2 0]
```



Numerical Representation of Tabular Data

Using One Hot Encoding:

```
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer

one_hot_encoder = ColumnTransformer(
    [['one_hot', OneHotEncoder(), ['Property_Area']],
     remainder='drop'
])

one_hot_encoded_column = one_hot_encoder.fit_transform(home_loan_prediction)
print(one_hot_encoded_column)
```

```
[[0. 0. 1.]
 [1. 0. 0.]
 [0. 0. 1.]
 ...
 [0. 0. 1.]
 [0. 0. 1.]
 [0. 1. 0.]]
```



Numerical Representation of Tabular Data

Using ordinal encoding:

```
ordinal_mapping = {"Urban": 0, "Rural": 1, "Semiurban": 2} # assumed hierarchy  
ordinal_encoded_column = home_loan_prediction['Property_Area'].map(ordinal_mapping)  
ordinal_encoded_column
```

Property_Area	
0	0
1	1
2	0
3	0
4	0
...	...
609	1
610	1
611	0

Numerical Representation of Tabular Data

Using target encoding with 'Loan Amount Term' as the target variable.

```
import numpy as np
target_encoding = home_loan_prediction.groupby('Property_Area')['Loan_Amount_Term'].mean().to_dict()
target_encoded_column = home_loan_prediction['Property_Area'].map(target_encoding)
target_encoded_column
```

	Property_Area
0	332.800000
1	345.531429
2	332.800000
3	332.800000
4	332.800000
...	...
609	345.531429

Numerical Representation of Tabular Data

Let's say we're predicting whether students pass an exam (1 = pass, 0 = fail), and we have a categorical column for "Subject". Here's the data:

Subject	Passed
Math	1
Science	0
Math	0
Science	1
History	1

Numerical Representation of Tabular Data

Step 1: Calculate the mean for each category

For each subject, we calculate the average of the "Passed" column:

- Math: $(1 + 0) / 2 = 0.5$
- Science: $(0 + 1) / 2 = 0.5$
- History: $(1) = 1.0$

Numerical Representation of Tabular Data

Step 2: Replace categories with these averages

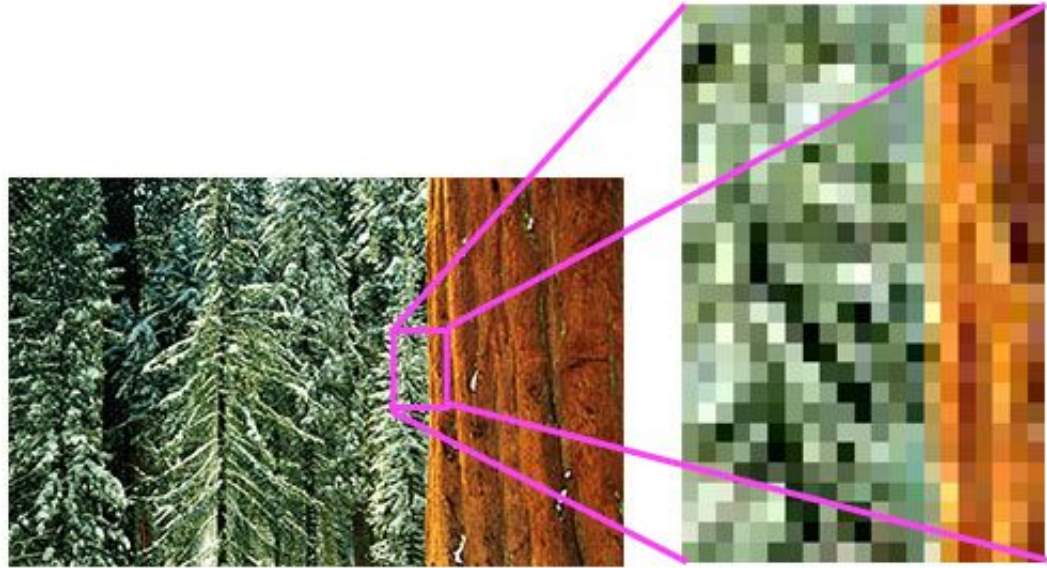
Now, we replace the subject names with the calculated mean values:

Subject	Passed	Target Subject Encoded
Math	1	0.5
Science	0	0.5
Math	0	0.5
History	1	1
Science	1	0.5

Image

Numerical Representation of Image Data

To understand how to represent an image in a numerical format, we need to understand what makes an image. The smallest building block of an image is called a pixel.



Numerical Representation of Image Data

The size of an image is described by the number of pixels in the height and the width.

Example: 1920x1080. This means that the screen will have a **width** of 1,920 pixels while the **height** of the screen will be 1,080 pixels. This results in a grand total of 2,073,600 pixels on-screen.

Each pixel has an **intensity value** that describes the intensity of color at that pixel.

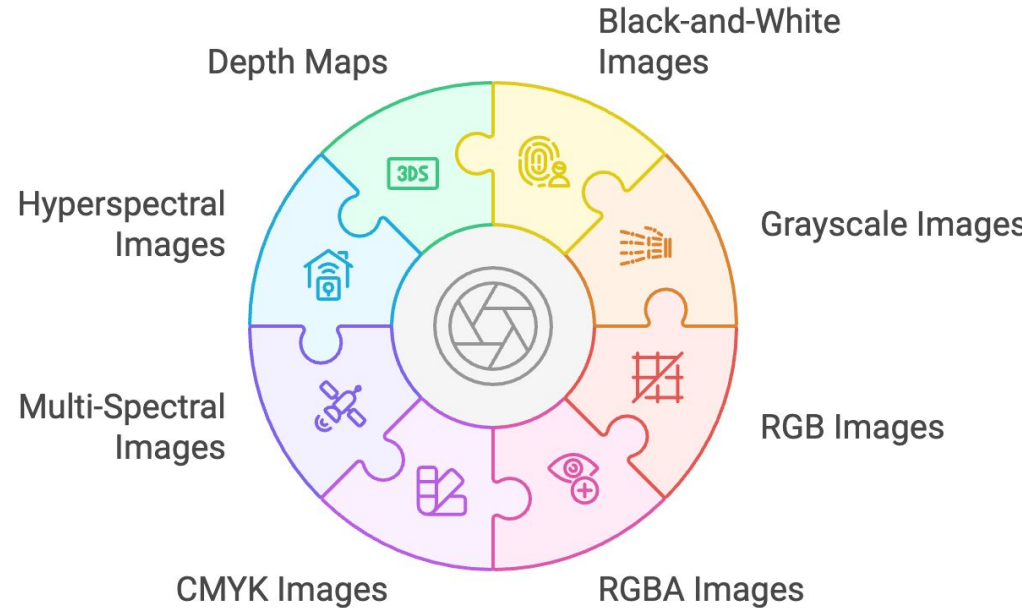
Numerical Representation of Image Data

Image Types and Machine Learning

The **matrix of pixel intensities** in an image is used as a numeric representation of that image.

The range of an image's pixel intensity depends on the **image type**.

We will go through the main image types: Black and white, Grayscale, RGB, RGBA, Multi-Spectral, and Depth Maps



Manipulating Image Data: OpenCV Library

To visualize the pixel values, we will use the opencv library to load the image and print its values.

For readability, we will resize the images to a smaller size.

```
import cv2
image = cv2.imread('path')
image = cv2.resize(image, (15, 15))
image
```

Numerical Representation of Image Data

Black and white images: Pixels are either 0 (black) or 1 (white), representing a binary image.



```
image = cv2.imread('/content/dino.png', cv2.IMREAD_GRAYSCALE)
image = cv2.resize(image, (15, 15))
_, binary_image = cv2.threshold(image, 100, 1, cv2.THRESH_BINARY)
binary_image

ndarray (15, 15) 
array([[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
       [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
       [1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1],
       [1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1],
       [1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1],
       [1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1],
       [1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1],
       [1, 1, 1, 1, 0, 1, 0, 0, 0, 0, 0, 1, 1, 1, 1],
       [1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1],
       [1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1],
       [1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1],
       [1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1],
       [1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1],
       [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
       [1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1],
       [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]], dtype=uint8)
```

Numerical Representation of Image Data

Grayscale Images: Pixel values range from 0 (black) to 255 (white) for 8-bit images. Higher bit-depth grayscale images (e.g., 16-bit) extend this range.

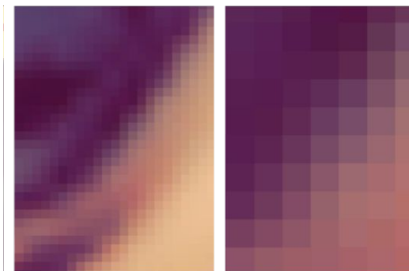
```
image = cv2.imread('/content/dino.png', cv2.IMREAD_GRAYSCALE)
image = cv2.resize(image, (15, 15))
image
```

```
ndarray (15, 15) hide data
array([[255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255],
       [255, 255],
       [255, 255, 254, 254, 254, 254, 254, 254, 254, 254, 254, 254, 254, 254, 254],
       [254, 255],
       [255, 255, 255, 255, 255, 255, 255, 255, 2, 0, 0, 0, 255, 255],
       [255, 255],
       [255, 255, 255, 255, 255, 255, 255, 255, 0, 0, 0, 0, 255, 255],
       [255, 255],
       [255, 255, 255, 255, 255, 255, 255, 255, 0, 0, 0, 0, 255, 255],
       [255, 255],
       [255, 255, 255, 255, 255, 255, 255, 255, 0, 0, 87, 255, 255, 255],
       [255, 255],
       [255, 255, 255, 255, 255, 255, 255, 253, 0, 0, 255, 255, 255, 255],
       [255, 255],
       [255, 255, 255, 255, 0, 255, 0, 0, 0, 1, 255, 255, 255, 255],
       [255, 255],
       [255, 255, 254, 254, 0, 0, 0, 0, 0, 25, 254, 254, 254, 254],
       [254, 255],
       [255, 255, 255, 255, 0, 0, 0, 0, 0, 187, 255, 255, 255, 255],
       [255, 255],
       [255, 255, 255, 255, 255, 0, 0, 0, 0, 255, 255, 255, 255, 255],
       [255, 255],
       [255, 255, 255, 255, 255, 251, 0, 253, 255, 255, 255, 255, 255, 255],
       [255, 255],
       [255, 255, 255, 255, 255, 255, 148, 255, 255, 255, 255, 255, 255, 255],
       [255, 255],
       [255, 255, 255, 255, 255, 255, 0, 255, 0, 255, 255, 255, 255, 255],
       [255, 255],
       [255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255],
       [255, 255],
       [255, 255]]), dtype=uint8)
```

Numerical Representation of Image Data

RGB images: each pixel in an RGB image can be represented as three values: Red, Green, and Blue.

Thus, RGB images can be represented as three channels. Each channel represents one red, green, and blue intensities respectively.



#642b4e R: 100 G: 43 B: 78	#7b4360 R: 123 G: 67 B: 96	#936073 R: 147 G: 96 B: 115
#7a4360 R: 122 G: 67 B: 96	#a1727a R: 161 G: 114 B: 122	#c89c8f R: 200 G: 156 B: 143
#945f71 R: 148 G: 95 B: 113	#ca9b91 R: 202 G: 155 B: 145	#f6d0ac R: 246 G: 208 B: 172



100	123	147
122	161	200
148	202	246



43	67	96
67	114	156
95	155	208



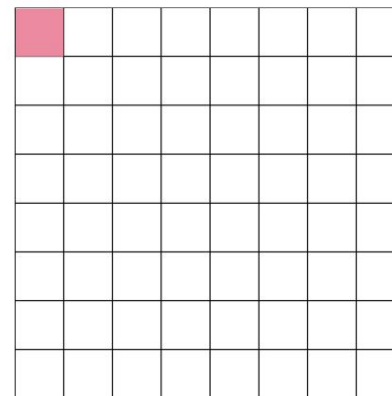
78	96	115
96	122	143
113	145	172

Numerical Representation of Image Data

RGBA images: Similar to RGB but with an additional alpha channel for transparency, where the alpha value typically ranges from 0 (fully transparent) to 255 (fully opaque).

index: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, ...

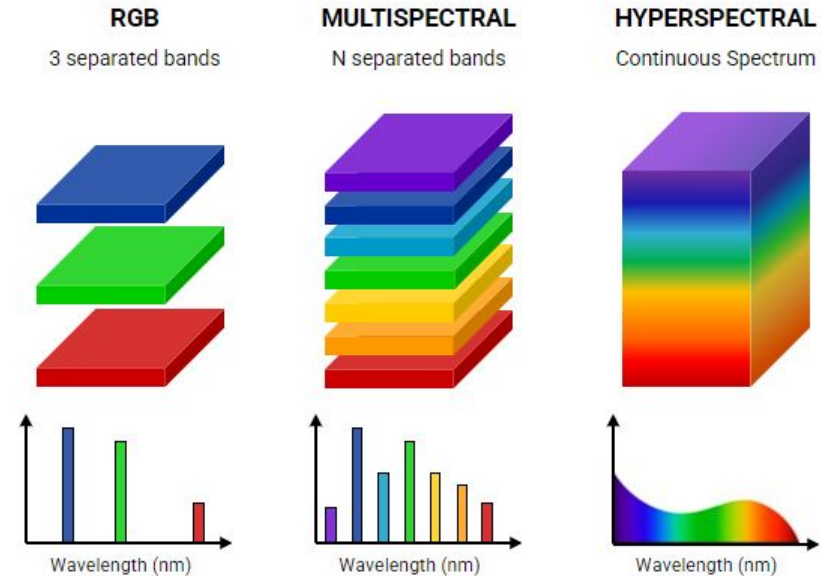
[r, g, b, a, r, g, b, a, r, g, b, a, ...]



Numerical Representation of Image Data

Multi-Spectral images: Images captured across multiple wavelengths of the electromagnetic spectrum, typically beyond the standard visible light range (red, green, and blue).

Commonly include 4–12 spectral bands, compared to the 3 bands (RGB) in standard images. Examples: Visible (RGB), near-infrared (NIR), short-wave infrared (SWIR), and ultraviolet (UV).



Numerical Representation of Image Data

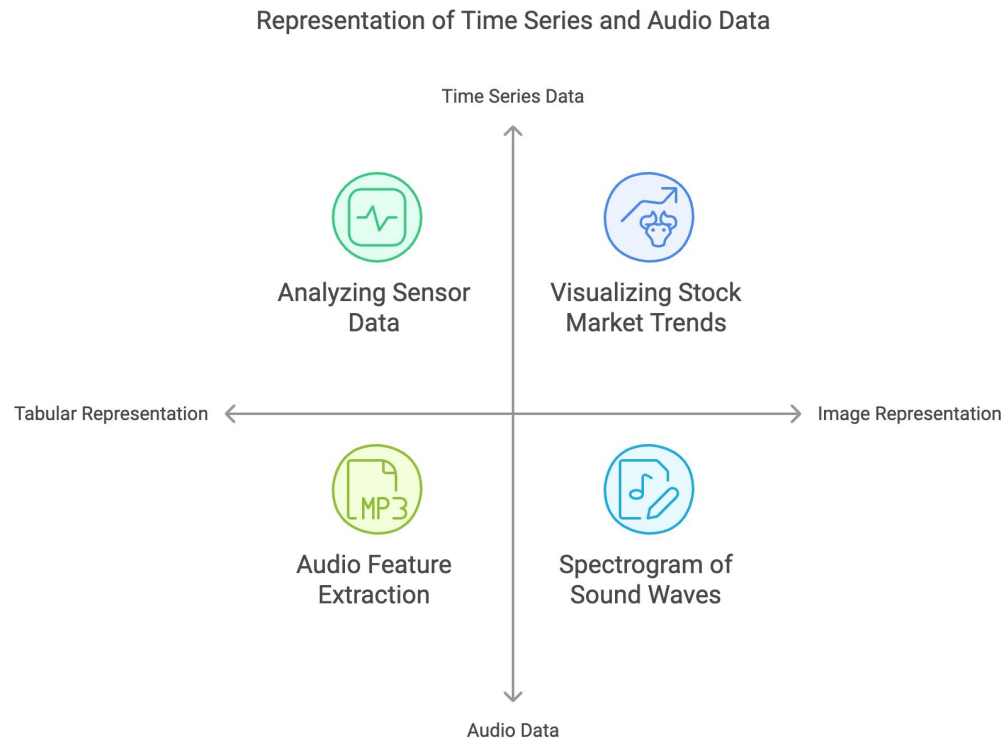
Depth Maps: A representation of the distances between the camera (or sensor) and objects in a scene, where each pixel's value corresponds to the depth (or distance) of that point. These maps are typically grayscale images where brighter pixels represent objects closer to the camera, and darker pixels represent objects farther away.



Audio and Time Series

Numerical Representation: Audio & Time Series Data

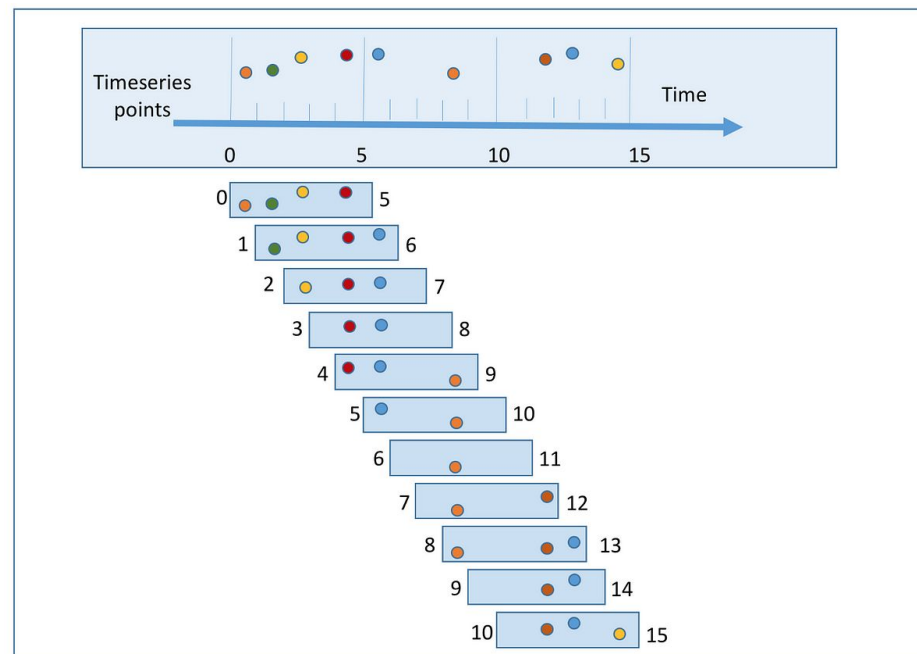
Depending on your application, time series and audio data can be transformed to either an **image** representation or **tabular** representation.



Numerical Representation: Audio & Time Series Data

Sliding Window: An approach used to capture discrete instances of continuous data that can be used for image or tabular transformation.

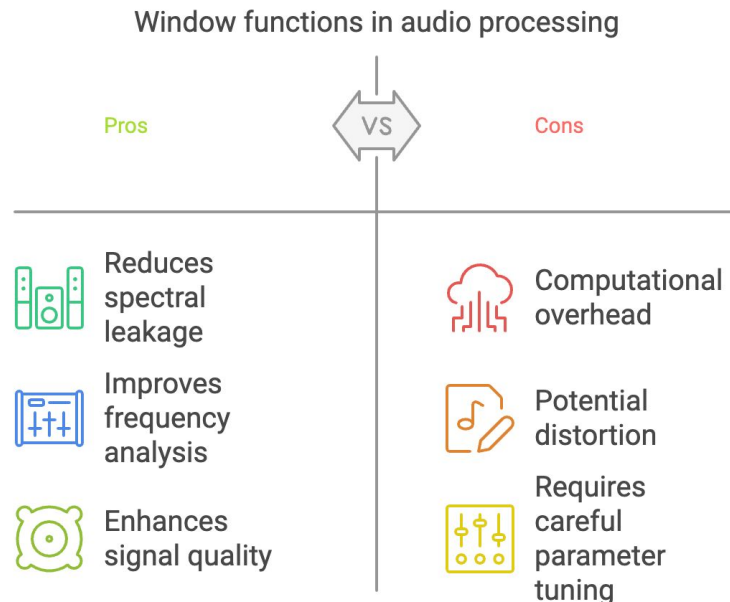
After segmenting the data into windows, a 'window function' is applied to each segment to **reduce the edge effects** (discontinuities) that would otherwise be introduced by dividing the signal into chunks.



Numerical Representation: Audio & Time Series Data

A window function is a mathematical function that is **applied to each segment** of the audio signal before performing any analysis (like Fourier Transform). The purpose of the window function is to smooth the signal at the edges of each segment, **reducing the abrupt discontinuities**.

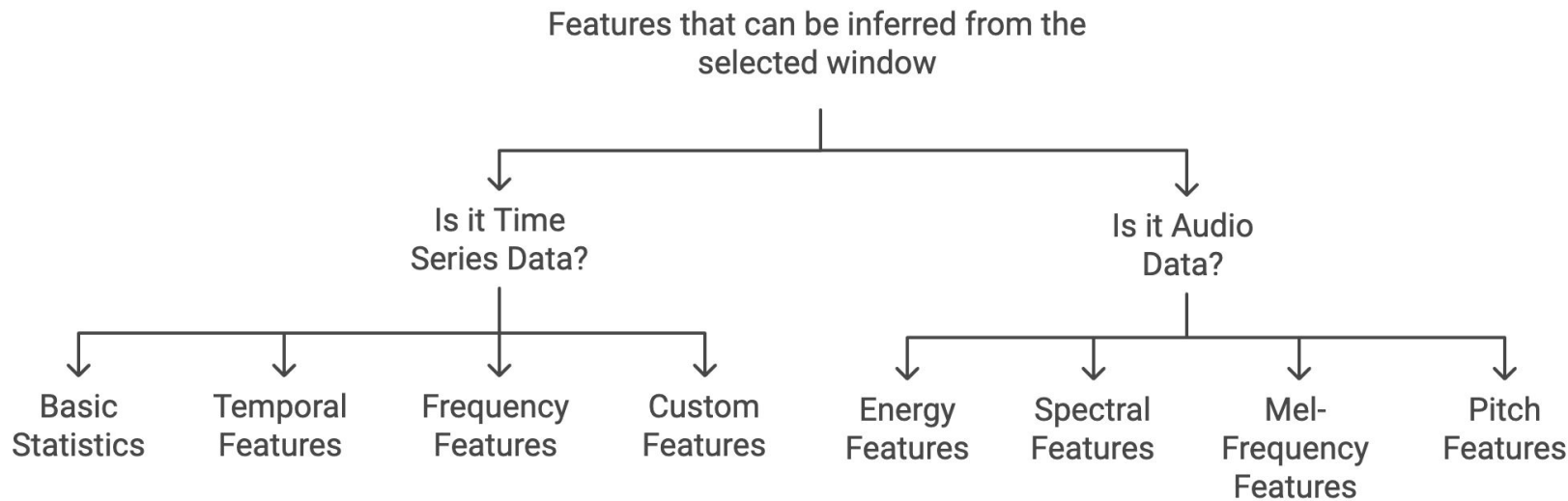
This minimizes the **spectral leakage** (energy spreading into other frequency bins) that results from discontinuities at the edges of the windows.



Numerical Representation: Audio & Time Series Data

Tabular Representation

Tabular representation of Audio and Time series data can be achieved by documenting the values of each window as a row in a structured representation.



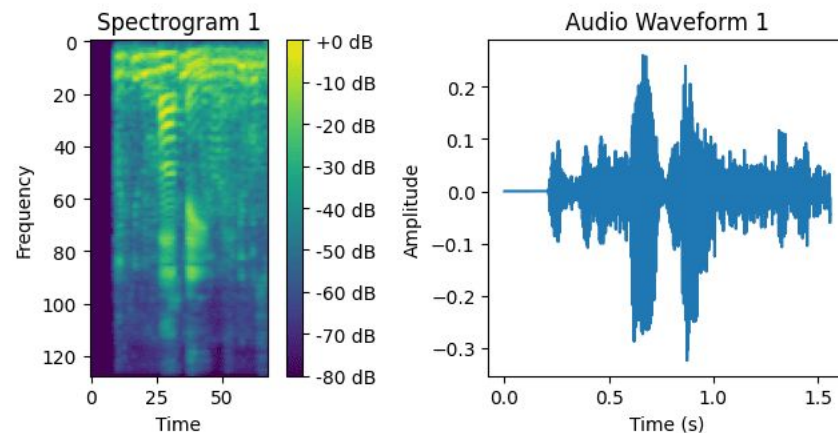
Numerical Representation: Audio & Time Series Data

Image Representation

Spectrograms are created by:

1. Applying **Short Time Fourier Transform** on the window after the window function has been applied.
2. Then, STFT is converted to a **log scale** to better match human hearing sensitivity (we are more sensitive to changes in lower frequencies than higher ones).
3. The spectrogram is then created by **stacking the frequency content of each windowed segment along the time axis**.

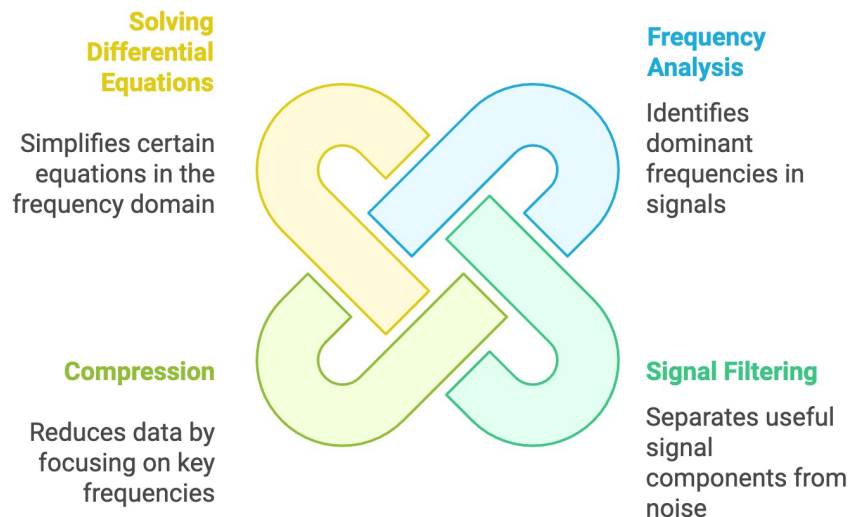
Spectrogram Comparison for Class: puppy



Reminder of Fourier Transform

The Fourier Transform (FT) **converts a signal from the time domain** (how a signal changes over time) **to the frequency domain** (what frequencies are present in the signal and their amplitudes).

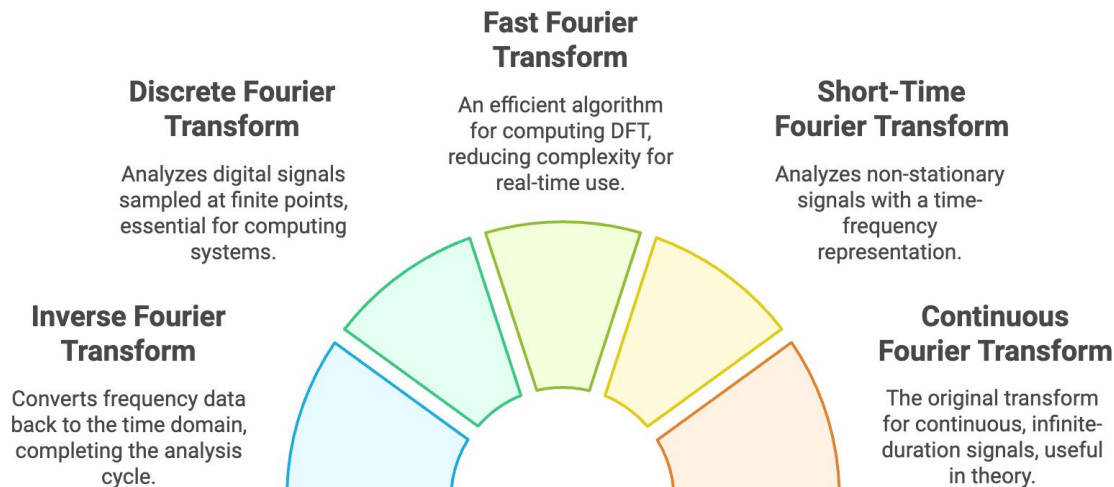
Applications of Fourier Transform



Reminder of Fourier Transform

Fourier transform variations:

Fourier Transform Variations

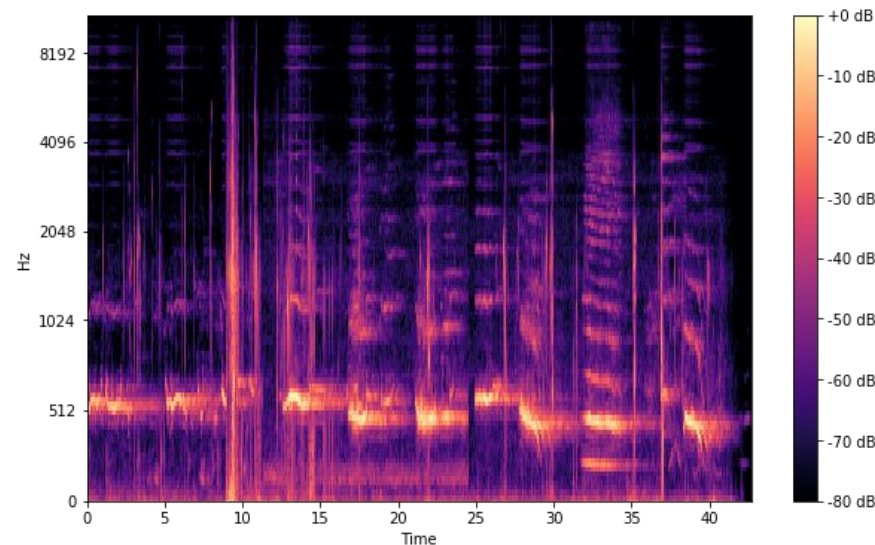


Numerical Representation: Audio & Time Series Data

Image Representation

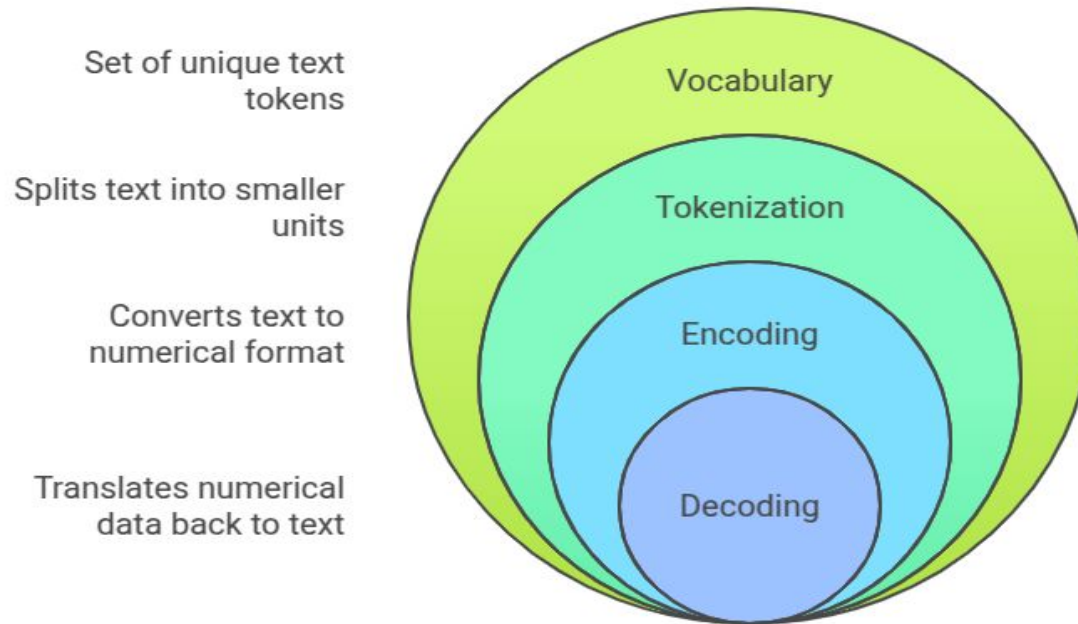
A more advanced version of the spectrogram is the Mel-frequency image.

1. Compute the spectrogram of the signal.
2. Apply the Mel Filter Bank that divides the frequency range (e.g., 0–8000 Hz for a 16 kHz signal) into N Mel bands (e.g., 40 bands).
3. For each Mel band, compute the power (square of the magnitude) of the frequencies within that band.
4. Plot the log power values of the Mel bands over time to create a 2D image



Text

Text Preprocessing Components



Numerical Representation of Text Data

Before we transform text into its numeric format, we need to **pre-process** the text:

Mandatory pre-processing:

1. Tokenization
2. Vocabulary Building

Optional pre-processing:

1. Remove links, tags, emojis, stop words, extra spaces and punctuations.
2. Transform all text into lower case.
3. Replace accented characters, numbers, and special characters with a different representation.

Numerical Representation of Text: Tokenization

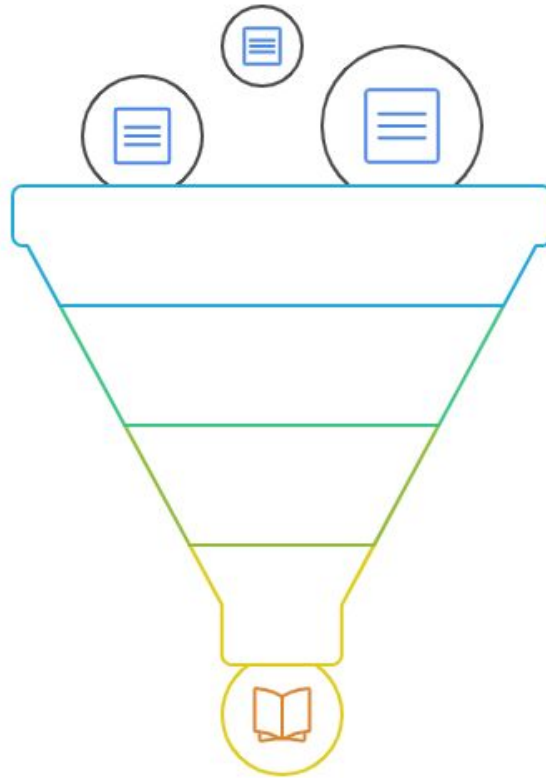
Tokenization: The process of **segmenting text into smaller, discrete units called tokens**, such as words, subwords, or characters. These tokens serve as the **basic building blocks for representing text** in a machine learning model, where each token is mapped to a unique index or representation.



Numerical Representation of Text: Tokenization

5 main types of tokenization are used to process text for natural language models:

1. **Character** level tokenization
2. **Word** level tokenization
3. **Byte** level tokenization
4. **Sub-word** level tokenization
5. **Sentence** level tokenization



Vocabulary Creation

Establishing a set of unique tokens



Tokenization

Splitting text into smaller units



Encoding

Converting text to numerical format



Decoding

Translating numbers back to text

Overview of Tokenization Methods

Byte Pair Encoding (BPE)

1. Initialize the text sequence

- Example: "banana" → **b a n a n a**

2. Identify the most frequent adjacent pair

- The pair "a n" appears most frequently.

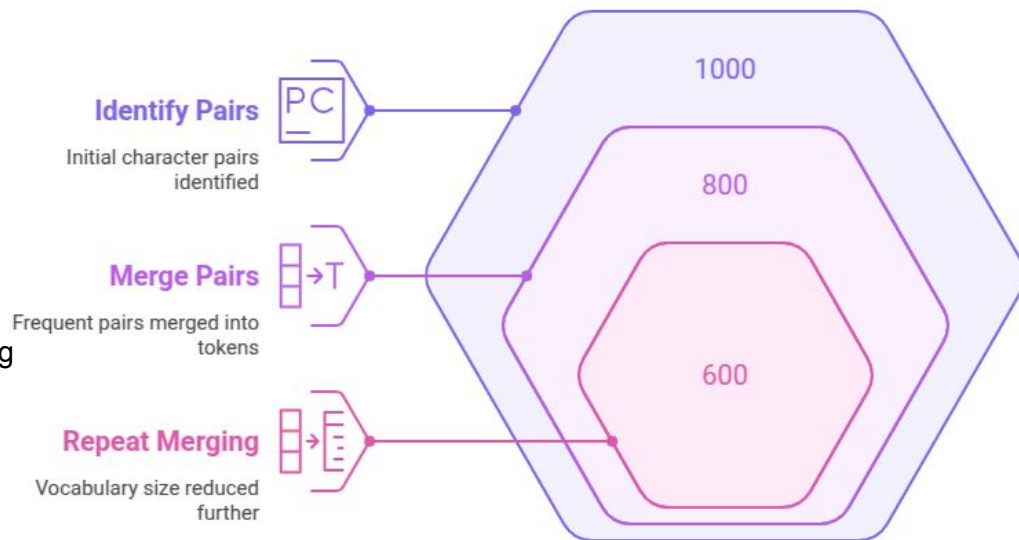
3. Merge the selected pair

- Updated sequence: **b AN a n a**

4. Repeat the merging process

- Final sequence: **b AN AN a**

Result: The text is tokenized into fewer subword units, reducing vocabulary size while preserving structure.



WordPiece Tokenization

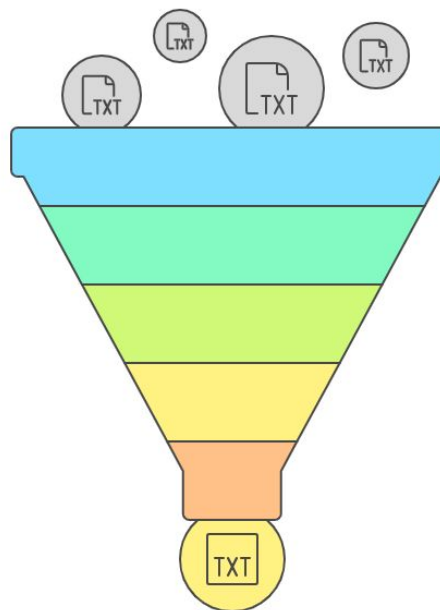
1. **Input word:** "unhappiness"

1. **Split into subwords:** ["un", "happiness"]

1. **Check vocabulary:** If "happiness" is not found, split further.

1. **Select tokens:** ["un", "happi", "###ness"] ("###" indicates a continuation).

1. **Final tokens:** ["un", "happi", "###ness"], reducing vocabulary size while preserving meaning.



Subword Splitting

Dividing text into smaller units



Likelihood Evaluation

Computing subword combination probabilities



Token Selection

Choosing the most probable sequence



Vocabulary Optimization

Adjusting vocabulary for efficiency



Efficient Token Representation

Finalizing minimal yet meaningful tokens

BERT Tokenization

Input Sentence:

"Playing outside is fun."

Step 1: Apply WordPiece Tokenization

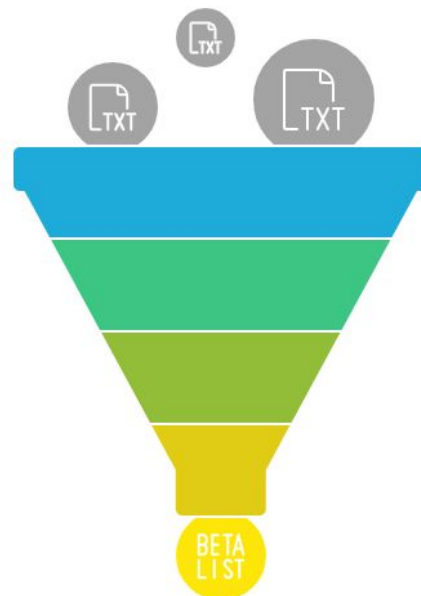
- "Playing" → **["Play", "##ing"]** (split into subwords)
- "outside" → **["outside"]** (kept as is)
- "is" → **["is"]** (kept as is)
- "fun" → **["fun"]** (kept as is)

Step 2: Add Special Tokens

- **[CLS]** at the beginning
- **[SEP]** at the end

Final Tokenized Output:

["[CLS]", "Play", "##ing", "outside", "is", "fun", "[SEP]"]



Subword Splitting

Words are divided into smaller units



Special Token Addition

Essential tokens for BERT functionality



Unknown Word Handling

Management of rare or unknown words



Contextual Embedding

Final tokens used for embeddings

Code Reference

Numerical Representation of Text: Tokenization

Python implementation of character level tokenization.

```
def character_tokenize(text):  
    return list(text)  
  
text = "Tokenization is essential. 😊"  
character_tokens = character_tokenize(text)  
print(character_tokens)  
  
['T', 'o', 'k', 'e', 'n', 'i', 'z', 'a', 't', 'i', 'o', 'n', ' ', 'i', 's', ' ', 'e', 's', 's', 'e', 'n', 't', 'i', 'a', 'l', '.', ' ', '😊']
```

Numerical Representation of Text: Tokenization

Python implementation of byte level tokenization.

```
def byte_level_tokenize(text):  
    return [c for c in text.encode('utf-8')]  
  
text = "Tokenization is essential. 😊"  
byte_tokens = byte_level_tokenize(text)  
print(byte_tokens)  
  
[84, 111, 107, 101, 110, 105, 122, 97, 116, 105, 111, 110, 32, 105, 115, 32, 101, 115, 115, 101, 110, 116, 105, 97, 108, 46, 32, 240, 159, 152, 132]
```

Some characters are represented by multiple bytes, like emojis.

```
print(f"length of byte tokenization: {len(byte_tokens)} \nlength of character tokenization {len(character_tokens)}")  
  
length of byte tokenization: 31  
length of character tokenization 28
```


Numerical Representation of Text: Tokenization

Python implementation of word level tokenization.

Different methods and libraries might tokenize the same string differently based on the pre-processing techniques used.

```
import nltk
nltk.download('punkt_tab')
from nltk.tokenize import word_tokenize
text = "Tokenization is essential. 😊"

def word_tokenize_nltk(text):
    return word_tokenize(text)

word_tokens_nltk = word_tokenize_nltk(text)
print(f"NLTK word level tokenization {word_tokens_nltk}")

from spacy.lang.en import English

nlp = English()
def word_tokenize_spacy(text):
    return [token.text for token in nlp(text)]

word_tokens_spacy = word_tokenize_spacy(text)
print(f"Spacy word level tokenization {word_tokens_spacy}")

import re

def word_tokenize_regex(text):
    return re.findall(r'\b\w+\b', text)

word_tokens_regex = word_tokenize_regex(text)
print(f"Regex word level tokenization {word_tokens_regex}")

[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt_tab.zip.
NLTK word level tokenization ['Tokenization', 'is', 'essential', '.', '😊']
Spacy word level tokenization ['Tokenization', 'is', 'essential', '.', '😊']
Regex word level tokenization ['Tokenization', 'is', 'essential']
```



Numerical Representation of Text: Tokenization

A unigram language model is trained on a corpus of data and can be loaded and used for tokenization.

```
import sentencepiece as spm

# Load a pre-trained SentencePiece model
sp = spm.SentencePieceProcessor(model_file='/content/test_model.model')

# Encode a sentence into subword strings
encoded_strings = sp.encode('This is a test', out_type=str)
print(encoded_strings)

['_This', '_is', '_a', '_', 't', 'est']
```

Numerical Representation of Text: Tokenization

Sentence Tokenizers are straightforward, where they are either rule based like NLK, Spacy, Regex, and CoreNLP. Otherwise, a machine learning model can be trained to predict the boundaries of sentences since rule-based approaches might become too nuanced.



Numerical Representation of Text: Vocabulary Building

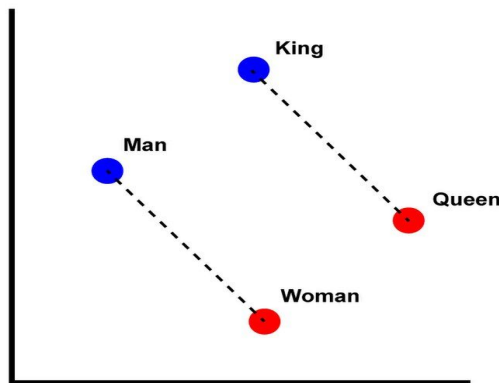
Special tokens are added to the vocabulary to account for different cases:

1. <UNK>: for unknown words (words that did not occur in the training corpus)
2. <STRT>: for tokens at the beginning of sentences
3. <END>: for the last token in a sentence or paragraph.
4. </W>: in case of byte, character, or sub-word encoding, this token is used to indicate the end of a word.

There are many more special sequences that we will come across in the upcoming chapters.

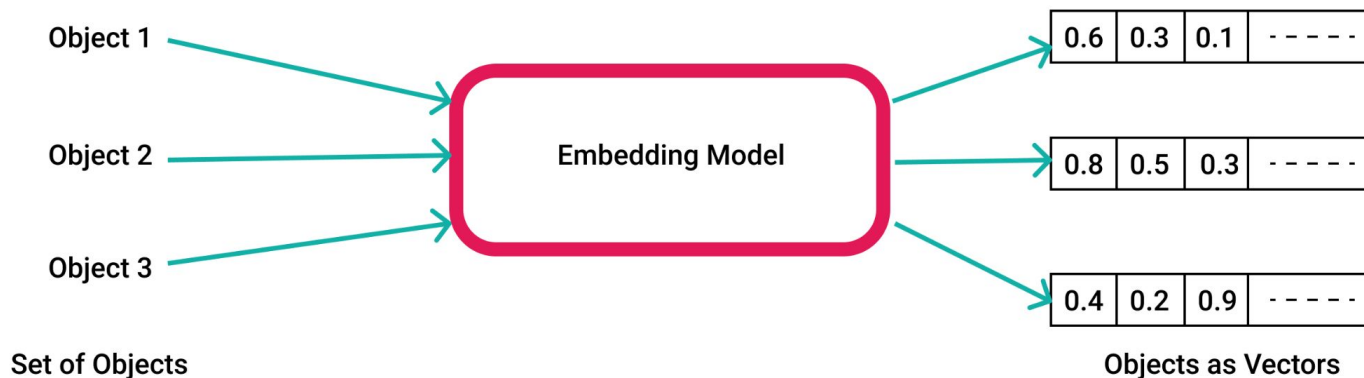
Numerical Representation of Text Data

A vector, in computer science, is an array of numbers designed to be expressive and feature-rich, effectively representing real-world objects or concepts in a structured numerical format, that lives in some space $\mathbb{R}^d$.



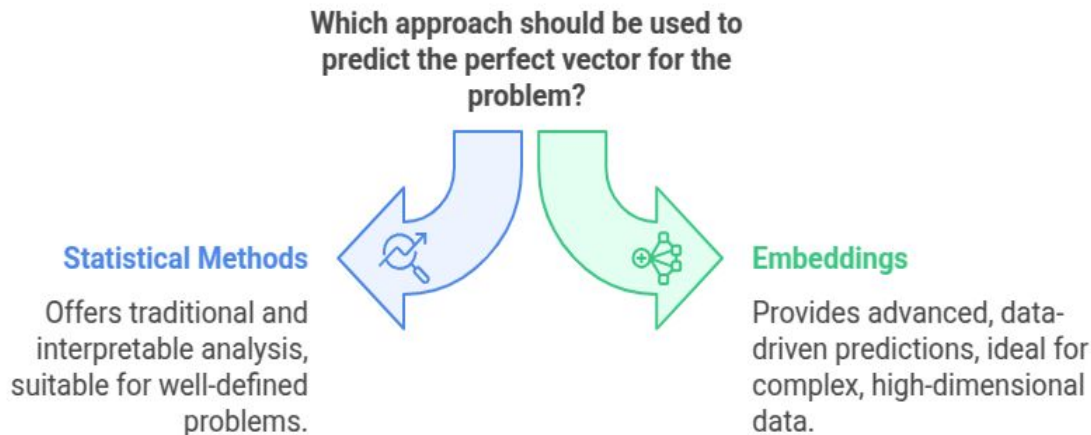
Numerical Representation of Text Data

A vector appears far more expressive than a simple ID in a vocabulary, as it provides a nuanced representation of a word. The key question, which we will briefly state now and elaborate on later, is: how do we effectively form such a vector?



Numerical Representation of Text Data

Using Statistical Methods or ML Based Models to predict the perfect vector for my problem.



Code Reference

(BOW, TFIdf, and Word2Vec)



Numerical Representation of Text Data: Vectorization

Bag of Words

```
from collections import Counter

def bag_of_words(text):
    # Tokenize the text (you can use any tokenization method you prefer)
    tokens = word_tokenize_regex(text.lower()) # Convert to lowercase for consistency

    # Remove punctuation and other non-alphanumeric characters
    tokens = [token for token in tokens if token.isalnum()]

    # Count the frequency of each word
    word_counts = Counter(tokens)

    return word_counts

# Example usage
text = "This is a sample text. This text is used for bag of words demonstration."
word_counts = bag_of_words(text)
word_counts

Counter({'this': 2,
        'is': 2,
        'a': 1,
        'sample': 1,
        'text': 2,
        'used': 1,
        'for': 1,
        'bag': 1,
        'of': 1,
        'words': 1,
        'demonstration': 1})
```



Numerical Representation of Text Data: Vectorization

TF-IDF

```
def word_tokenize_regex(text):
    # Simple regex-based tokenizer
    return re.findall(r'\w+', text)

def calculate_term_frequency(token, tokens_in_document):
    # Calculate term frequency for a specific token in the document
    frequency_of_all_tokens = Counter(tokens_in_document)
    frequency_of_token = frequency_of_all_tokens[token]
    return frequency_of_token / len(tokens_in_document)

def calculate_inverse_document_frequency(token, corpus):
    # Calculate IDF for a specific token
    number_of_documents_that_contain_token = sum(1 for document in corpus if token in word_tokenize_regex(document.lower()))
    number_of_documents_in_corpus = len(corpus)
    return math.log(number_of_documents_in_corpus / (1 + number_of_documents_that_contain_token))

def tfidf(document, corpus):
    tokens_in_document = word_tokenize_regex(document.lower())
    tfidf_scores = {}
    for token in tokens_in_document:
        term_frequency = calculate_term_frequency(token, tokens_in_document)
        inverse_document_frequency = calculate_inverse_document_frequency(token, corpus)
        tfidf_scores[token] = term_frequency * inverse_document_frequency
    return tfidf_scores
```

Numerical Representation of Text Data: Embeddings

Word2Vec Model



```
from gensim.models import Word2Vec
# Sample text for demonstration
sentences = [
    ['this', 'is', 'a', 'sample', 'sentence'],
    ['another', 'sentence', 'for', 'demonstration'],
]

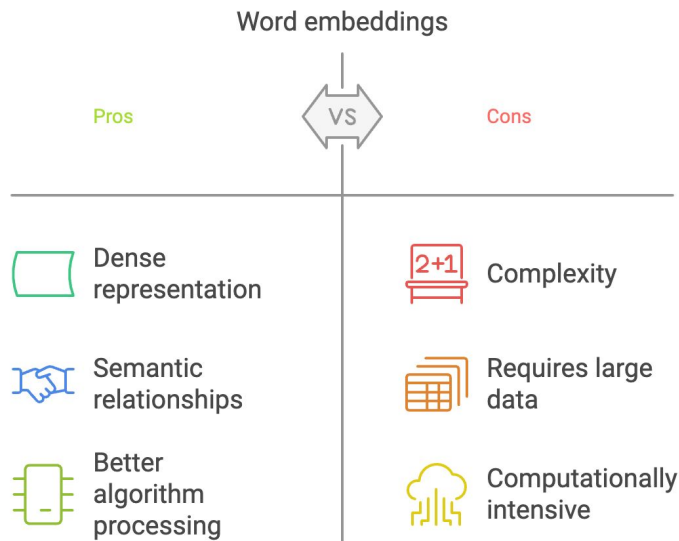
# Train a Word2Vec model (replace with your own model or data)
model = Word2Vec(sentences, vector_size=100, window=5, min_count=1, workers=4)

# Get the embedding for a word
word = 'sentence'
if word in model.wv:
    embedding = model.wv[word]
    print(f"Embedding for '{word}': {embedding}")
else:
    print(f"Word '{word}' not found in vocabulary.")
```

Numerical Representation of Text Data: Embeddings

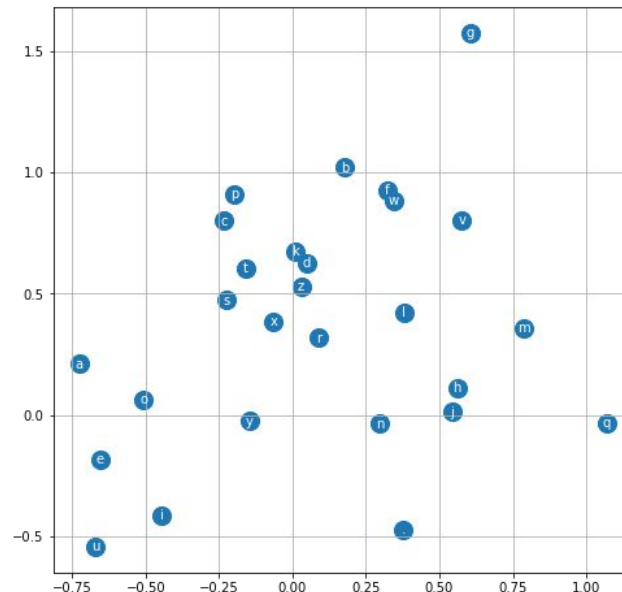
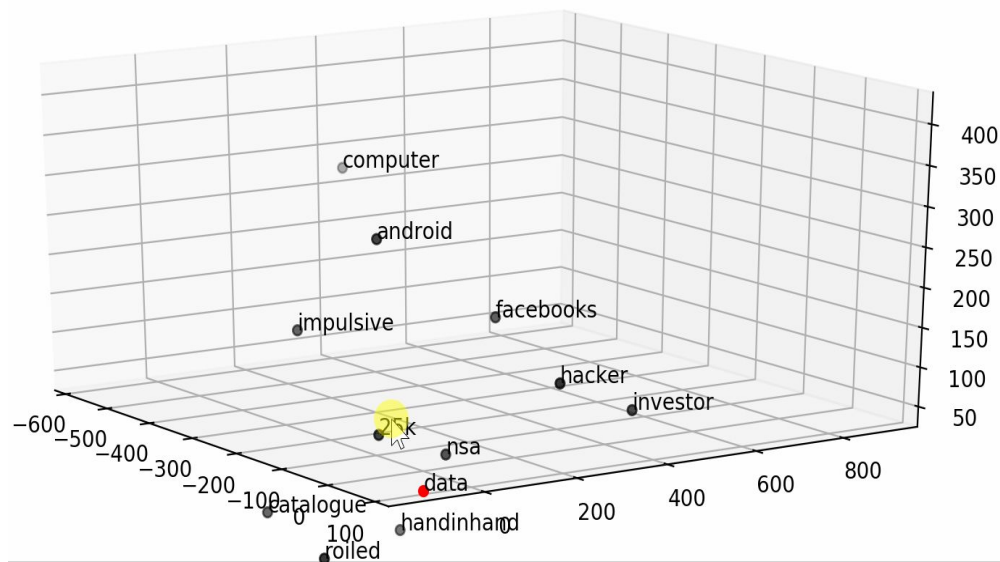


An embedding is a **dense**, **multidimensional** representation of a token that encodes **semantic**, **contextual**, and **statistical** information, capturing **relationships** and **patterns** within the data.



Numerical Representation of Text Data: Embeddings

Embeddings are **generated through machine learning models** that are trained specifically to output this numeric representation of text, so we can learn meanings and semantics of the tokens.



Numerical Representation of Text Data: Embeddings



AMERICAN
UNIVERSITY
OF BEIRUT

First Effective way to Learn Embeddings [2007, Bengio]

A Neural Probabilistic Language Model

Yoshua Bengio

Réjean Ducharme

Pascal Vincent

Christian Jauvin

Département d'Informatique et Recherche Opérationnelle

Centre de Recherche Mathématiques

Université de Montréal, Montréal, Québec, Canada

BENGIOY@IRO.UMONTREAL.CA

DUCHARME@IRO.UMONTREAL.CA

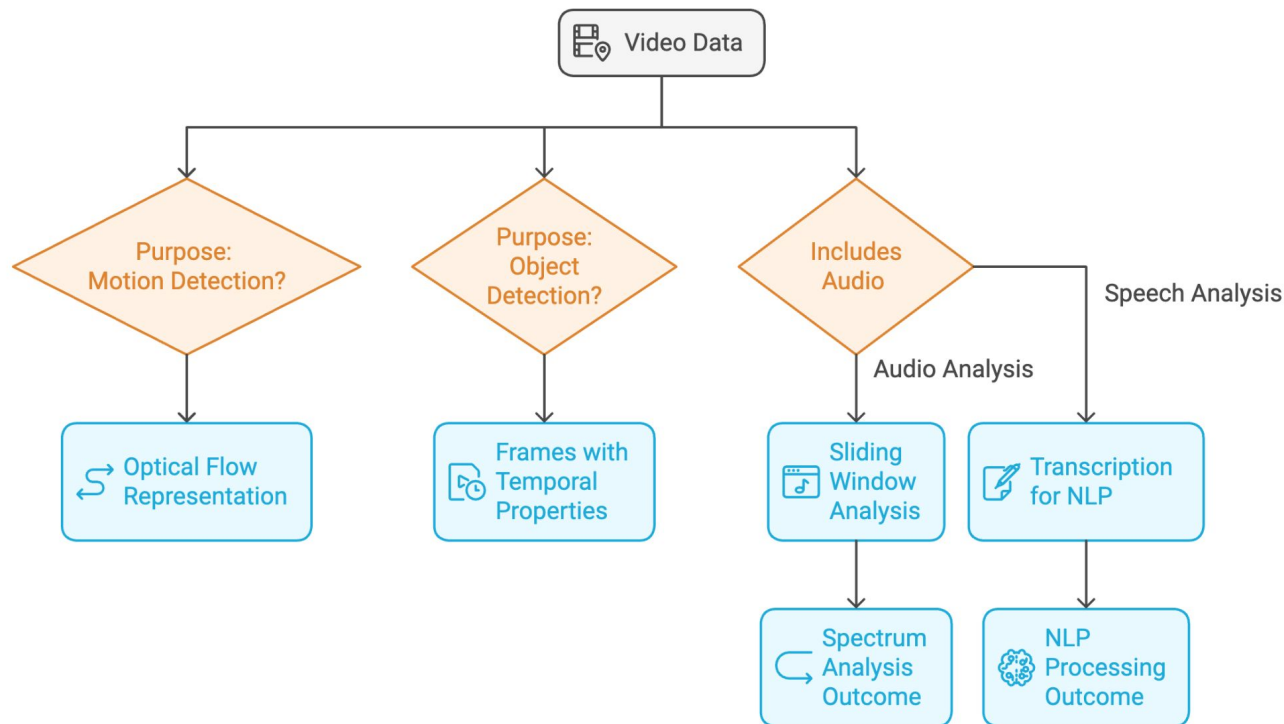
VINCENTP@IRO.UMONTREAL.CA

JAUVINC@IRO.UMONTREAL.CA

Editors: Jaz Kandola, Thomas Hofmann, Tomaso Poggio and John Shawe-Taylor

Video Data

Numerical Representation of Video Data





Thank You